



Claude Code · SONSLAB

Claude Code 한글 가이드북

처음부터 실전까지 · 한 권으로 끝내는 큐레이션 가이드

발행일	2026-04-26
버전	v2026.17-guidebook
출처	https://code.claude.com/docs
갱신 주기	매주 일요일 20:00 KST
발행처	SONSLAB · shonsubong.github.io/sonslab-blog

SONSLAB 로고는 무엇이든 답을 수 있는 비어 있는 손수레에서 모티브를 가져왔습니다.

들어가며

이 책은 Anthropic이 만든 터미널·IDE 통합 코딩 에이전트 Claude Code를 한국어로 처음부터 끝까지 안내합니다. Anthropic 공식 문서를 단순히 번역해 옮긴 게 아니라, 실제로 매일 쓰면서 부딪치게 되는 순서대로 다시 묶었습니다.

이 책의 원칙

- 독자를 먼저 생각합니다. 실무자가 다음에 무엇을 해야 하는지를 가장 먼저 보여줍니다.
- 가장 쓸모 있는 것을 위로. 설치 다음 페이지부터 바로 첫 명령을 칠 수 있도록 구성했습니다.
- 설명보다 예시. 모든 개념은 실제 명령·코드 한 토막과 함께 등장합니다.
- 최신 상태 유지. 매주 일요일 20:00 KST에 자동 갱신됩니다.
- 중복하지 말고 연결. 공식 문서를 베끼지 않고, 필요한 페이지로 링크합니다.

어떻게 읽으면 좋을까

처음 Claude Code 를 만진다면 Part 1 → Part 2 순서대로 한 번 훑은 뒤, 자기 프로젝트에서 한 시간 정도 가지고 놀아 보세요. 이미 쓰고 있다면 Part 3 (도구 확장) 와 Part 5 (운영) 만 골라 봐도 됩니다. 각 챕터 끝의 “더 알아보기” 박스가 공식 문서의 정확한 페이지로 안내합니다.

주의 · 본 가이드북은 SONSLAB이 정리한 비공식 문서입니다. 권한·보안·결제·법적 사안은 반드시 code.claude.com/docs 의 공식 문서를 1차 자료로 확인하세요.

목차

Part 1 — 시작하기 (초급)

- 1.1 Claude Code 란 무엇인가
- 1.2 설치
- 1.3 첫 실행과 프로젝트 이해
- 1.4 첫 5분 — 실전 워크플로
- 1.5 agent loop 이해하기
- 1.6 CLAUDE.md 첫 작성
- 1.7 흔한 막힘과 해결 (초급)

Part 2 — 일상 사용 (초급→중급)

- 2.1 CLAUDE.md 깊이 있게
- 2.2 메모리 시스템 — # 와 /memory
- 2.3 슬래시 명령
- 2.4 파일 편집 워크플로
- 2.5 터미널 통합
- 2.6 Plan 모드 깊이 있게
- 2.7 Git 통합
- 2.8 권한 설정 첫발

Part 3 — 도구 확장 (중급)

- 3.1 MCP — 외부 도구 연결 (개요)
- 3.2 자주 쓰는 MCP 톱 10
- 3.3 사용자 정의 MCP 서버 만들기
- 3.4 Skills — 특정 작업 모범 사례
- 3.5 Skills 만들기 (실전)
- 3.6 Plugins — 묶음 설치
- 3.7 플러그인 만들기·배포
- 3.8 Hooks — 자동 실행 후크
- 3.9 Subagents — 서브에이전트

Part 4 — Agent SDK (중급→고급)

- 4.1 Agent SDK 개요
- 4.2 Python SDK 빠른 시작
- 4.3 TypeScript SDK 빠른 시작
- 4.4 시스템 프롬프트 수정
- 4.5 사용자 정의 도구 추가
- 4.6 Streaming Output
- 4.7 Streaming Input
- 4.8 Structured Output
- 4.9 Tool Search
- 4.10 Channels — 외부에서 이벤트 밀어넣기
- 4.11 SDK 안에서의 Subagents
- 4.12 승인 흐름

Part 5 — 운영 (중급)

- 5.1 베스트 프랙티스
- 5.2 비용 관리
- 5.3 성능 튜닝
- 5.4 보안 기본
- 5.5 권한 정책 깊이

- 5.6 트러블슈팅
- 5.7 협업 워크플로
- 5.8 다른 환경 — Web · IDE · Chrome · Desktop

Part 6 — 고급 패턴 (고급)

- 6.1 서버에이전트 오케스트레이션
- 6.2 채널 + 서버에이전트로 라이브 시스템
- 6.3 마이그레이션·리팩터링 자동화
- 6.4 CI/CD 통합
- 6.5 보안 샌드박스
- 6.6 텔레메트리·모니터링
- 6.7 멀티프로젝트 워크플로
- 6.8 자가 페이싱 자동화 (Monitor + /loop)

Part 7 — 엔터프라이즈 (고급)

- 7.1 조직 배포 결정 트리
- 7.2 Bedrock / Vertex / Foundry 통합
- 7.3 SSO · SCIM 으로 좌석 관리
- 7.4 감사 로그·컴플라이언스
- 7.5 LLM Gateway
- 7.6 관리자 강제 정책 (Managed Settings)

부록

- 8.1 자주 쓰는 명령 빠른 참조
- 8.2 한·영 용어집 (확장판)
- 8.3 슬래시 명령 전체 목록
- 8.4 MCP 서버 카탈로그
- 8.5 환경 변수 참조
- 8.6 에러 코드 사전
- 8.7 학습 리소스
- 8.8 변경 이력 (2026-04 기준 주요)

Part 8 — 실전 시나리오 모음

- 9.1 시나리오 1 — 신입의 첫 주 (초급)
- 9.2 시나리오 2 — 한 주의 사이드 프로젝트 (초급→중급)
- 9.3 시나리오 3 — 운영 인시던트 대응 (중급)
- 9.4 시나리오 4 — 큰 마이그레이션 (중급→고급)
- 9.5 시나리오 5 — 사내 도구 자동화 (고급)
- 9.6 시나리오 6 — Hooks 로 정책 강제 (고급)
- 9.7 시나리오 7 — 멀티 에이전트로 신규 서비스 부트스트랩 (고급)
- 9.8 시나리오 8 — 클로드 코드와 함께하는 기술 면접 준비 (개인 활용)
- 9.9 시나리오 9 — 100% 한국어 프로젝트 운영
- 9.10 시나리오 10 — 본인의 학습 가속

Part 9 — 쿡북 (실전 레시피)

- 10.1 코드 이해 / 탐색
- 10.2 리팩터링
- 10.3 디버깅
- 10.4 테스트 작성
- 10.5 데이터베이스
- 10.6 보안
- 10.7 성능
- 10.8 문서
- 10.9 회의·소통

Part 10 — 자주 묻는 질문

- 11.1 비용
- 11.2 보안
- 11.3 권장 사용
- 11.4 학습

Part 11 — 프롬프트 템플릿 모음

- 12.1 코드 검토 템플릿
- 12.2 작업 분할 템플릿
- 12.3 학습 템플릿
- 12.4 운영 템플릿
- 12.5 디자인 템플릿
- 12.6 커뮤니케이션 템플릿
- 12.7 코드 작성 템플릿
- 12.8 자동화 템플릿
- 12.9 데이터 분석 템플릿
- 12.10 글쓰기 템플릿
- 12.11 회고 템플릿

Part 12 — 마무리 — 한 달 동안 만져 보기 가이드

- 13.1 주 학습 계획
- 13.2 한 달 후의 본인
- 13.3 추천 다음 단계
- 13.4 이 책에 대한 한 마디

PART 1

시작하기 (초급)

이 책의 첫 Part 는 "Claude Code 가 무엇이고 왜 쓰는지" 와 "지금 당장 첫 명령을 어떻게 쳐야 하는지" 두 가지에 답합니다. 이 Part 만 끝내도 본인 프로젝트에 Claude Code 를 붙여 한 시간 정도 직접 만져 볼 수 있는 수준이 됩니다. 한 시간 직접 만진 사람과 그렇지 않은 사람의 이해도는 다섯 배 차이가 납니다. 책을 끝까지 읽기 전에 여기까지만 마치고 한 번 시도해보세요.

Chapter 1.1

Claude Code 란 무엇인가

Claude Code 는 Anthropic 이 만든 터미널·IDE 통합 코딩 에이전트 입니다. 한 줄로 요약하면 "내 컴퓨터의 셸과 파일 시스템에 직접 손을 댈 수 있는 Claude" 입니다. 일반 챗봇과 다른 핵심 차이는 다음 셋 입니다.

첫째, 파일을 직접 편집합니다. 변경 사항을 텍스트로 출력해 사용자에게 복사·붙여넣기를 시키지 않습니다. `apps/web/src/auth.ts:42` 같은 정확한 위치를 짚어 패치를 적용하고, 적용 결과를 다시 읽어 본인 의도와 일치하는지 확인합니다.

둘째, 명령을 실행합니다. `npm test` 를 돌려 결과를 보고, 실패한 테스트 출력을 읽어 다음 수정을 결정합니다. 이 "쓰기 → 실행 → 결과 확인 → 다시 쓰기" 의 반복이 Claude Code 의 핵심 루프이고, 이 루프를 agent loop 이라고 부릅니다.

셋째, 장시간 작업을 분할해 끝까지 갑니다. "이 마이그레이션 끝까지 부탁한다" 같은 큰 요청을 여러 단계의 todo 로 쪼개고, 각 단계 결과를 확인해 가며 끝까지 진행합니다. 사용자는 마지막에 결과만 확인하면 됩니다.

요약하면 Claude Code 는 "API 한 번 호출 = 한 마디 응답" 의 전통적인 LLM 사용법이 아니라, 여러 차례의 도구 호출과 결과 합성을 거쳐 하나의 작업 단위를 끝내는 에이전트 입니다.

누구에게 적합한가

- 프로젝트의 코드 베이스가 큰 경우: 함수 이름·파일 위치를 일일이 알려주는 대신 "이 동작을 추가해 달라" 정도의 요구로 충분합니다.
- 반복 작업이 잦은 경우: 테스트·린트·빌드·배포 명령이 손에 익었다면 Claude 가 그 위에서 작동합니다.
- 혼자 작업하는 사이드 프로젝트: 페어 프로그래머 한 명이 합류한 효과가 가장 큽니다.
- 레거시 코드 베이스 탐색: 누가 짰 지 모르는 코드를 빠르게 이해할 때 압도적입니다.

적합하지 않은 경우

- 회사 보안 정책으로 외부 LLM 호출이 금지된 환경. 자세한 내용은 Part 5 의 "보안·권한" 챕터에서 다룹니다.
- 창의성·디자인이 80% 이상인 작업. 시각 디자인, 카피라이팅, 사용자 리서치 같은 것은 다른 도구가 더 낫습니다.
- 읽기 전용 검토만 필요한 경우. 그건 그냥 chat 인터페이스가 더 빠릅니다.

2026년 4월 기준 핵심 기능

- 터미널 CLI · IDE 확장 · 웹 · 데스크톱 앱 · Chrome 확장까지 다섯 가지 진입점
- Auto mode (안전 변경은 자동 승인, 위험 동작만 멈추는 중간 모드)
- Computer use (CLI 와 데스크톱 앱에서 실제 화면을 클릭/검증)
- Ultraplan (클라우드에서 계획을 세우고 브라우저로 검토)
- Monitor 도구 (백그라운드 워커가 이벤트를 세션에 흘려 넣음)
- `/autofix-pr` 로 터미널에서 PR 직접 자동 수정
- `/team-onboarding` 로 본인 설정을 한 묶음으로 패키징해 동료에게 배포
- 100 개 이상의 MCP 서버 (GitHub, Slack, Linear, Notion, Postgres 등)
- 마켓플레이스를 통한 플러그인 · 스킴 설치

Chapter 1.2

설치

시스템 요구

- macOS, Linux, Windows (네이티브 또는 WSL) 모두 지원합니다.
- Node.js 18 이상.
- Anthropic 계정 또는 Bedrock / Vertex / Foundry 자격 증명. 개인이라면 Claude Pro 또는 Max 구독으로 시작합니다.
- 디스크 약 200MB. 메모리는 일반 셸 세션 + 약 500MB.

설치 한 줄

```
npm install -g @anthropic-ai/claude-code
```

-g 플래그는 전역 설치를 의미합니다. **claude** 명령이 PATH 어디서나 실행되도록 합니다.

설치가 끝나면 버전을 확인하세요.

```
claude --version
```

버전이 나오면 끝입니다. **command not found** 가 나온다면 npm 의 글로벌 bin 디렉터리가 PATH 에 들어 있는지 확인해야 합니다 (**npm prefix -g** 로 위치를 확인). Windows 의 경우 **%APPDATA%\npm** 이 PATH 에 있는지 확인하세요.

Windows 사용자를 위한 추가 안내

Windows 에서는 두 가지 모드가 있습니다.

- WSL2 (강력 권장) — 우분투 안에서 위 npm 명령을 그대로 실행. 셸 동작·파일 권한·Git 사용성이 macOS/Linux 와 거의 동일.
- 네이티브 Windows — Node.js 가 깔려 있으면 위 npm 명령이 그대로 동작. 다만 셸은 cmd.exe 가 아닌 PowerShell 권장. 2026 년 3 월부터 PowerShell 도구가 Bash 와 동등 수준으로 지원됩니다.

WSL 안에서는 한국어 입력기·터미널 폰트가 가끔 깨지므로 Windows Terminal + WSL Ubuntu 조합을 추천합니다.

인증

처음 실행할 때 인증 흐름이 시작됩니다. 가장 흔한 경로 두 가지:

```
# 1) Claude Pro / Max 구독 (개인)
claude
# → 브라우저가 열리고 SSO 로그인을 마치면 토큰이 로컬에 저장됩니다.
```

```
# 2) API 키
export ANTHROPIC_API_KEY=sk-ant-xxxxx
claude
```

인증이 한 번 끝나면 다음 실행부터는 자동으로 토큰이 갱신됩니다. 토큰 저장 위치는 macOS·Linux 의 경우 `~/.claude/`, Windows 는 `%APPDATA%\Claude\` 입니다.

회사 계정 사용

Claude for Teams 또는 **Claude Enterprise** 를 쓰는 조직이라면 관리자가 SSO/SCIM 으로 좌석을 배정해 줍니다. 첫 로그인 시 회사 이메일을 입력하면 됩니다. AWS Bedrock, GCP Vertex AI, Azure Foundry 를 통한 호출은 Part 7 에서 자세히 다룹니다.

Chapter 1.3

첫 실행과 프로젝트 이해

설치한 폴더 또는 아무 프로젝트 루트에서 **claude** 한 줄을 입력하면 대화형 세션이 열립니다. 안으로 들어가면 시각적으로는 평범한 터미널이지만, 그 안에서 자연어로 부탁하면 Claude 가 파일을 편집하고 명령을 돌립니다.

> Explain what this repo does, in three short paragraphs.

이 한 줄로 Claude 는 README, package.json, 핵심 진입점을 스스로 골라 읽고 요약해 줍니다. 처음 만나는 코드 베이스에 적응할 때 가장 빠른 길이고, 동시에 Claude 의 동작 패턴 — "여러 파일을 자기가 골라 읽고 종합한다" — 을 한눈에 보여 주는 예시입니다.

왜 이 첫 명령을 추천하는가

- Claude 가 코드 베이스를 자기 시각으로 어떻게 읽는지 확인할 수 있습니다.
- 잘못 이해한 부분이 있다면 곧바로 교정할 수 있습니다 ("아니, 이 모듈은 사실...").
- 코드 베이스의 첫 인상이 잡히면 이후 작업 요청이 훨씬 정확해집니다.

흔한 첫날 시도

- "README 를 읽고 5분 안에 시작할 수 있게 다시 정리해 줘"
- "이 프로젝트의 주요 의존성 5개와 각각의 역할을 표로"
- "테스트는 어디에 있고 어떤 명령으로 돌리는지"
- "최근 한 달 동안 가장 많이 변경된 파일 톱 10"

Chapter 1.4

첫 5분 — 실전 워크플로

설치가 끝났다면 다음 다섯 명령을 직접 쳐 보세요. 5분 안에 Claude Code 의 사용법이 손에 들어옵니다.

1) 프로젝트 이해

> What does this repo do? List the top 5 important files.

2) 작은 변경

> Add a /healthz endpoint to the API that returns {ok: true}. Wire it into the existing router.

Claude 는 라우터 파일을 찾아 추가 위치를 정하고, 기존 핸들러 패턴을 따라 새 핸들러를 작성한 뒤 다시 보여 줍니다.

3) 테스트 실행

> Run the existing tests to make sure nothing broke.

테스트가 실패하면 Claude 가 출력을 직접 읽고 원인을 추정해 다시 수정합니다.

4) 커밋 메시지

> Stage the changes and write a one-line commit message in the imperative mood.

5) 다음 일감

> What did we just change? Anything we should clean up before the PR?

이 다섯 단계가 일상 워크플로의 기본 골격입니다. 처음에는 명령을 하나씩 실행해 결과를 확인하고, 익숙해지면 "이 기능 끝까지 부탁" 같은 큰 단위로 묶어 던지게 됩니다.

첫날에 자주 하는 실수

- 너무 큰 일감을 한 번에 던진다. "이 SaaS 를 처음부터 새로 만들어 줘" 보다 "사용자 모델을 추가하고 회원가입 라우트를 붙여 줘" 가 훨씬 잘 됩니다.
- 결과를 확인하지 않고 다음을 던진다. Claude 의 변경을 한 번 읽고 의도와 맞는지 보세요. 중간에 의도가 어긋나기 시작하면 그 자리에서 교정해야 합니다.
- 자기 코드 컨벤션을 안 알려 준다. 들여쓰기·이름 규칙·테스트 패턴이 비표준이라면 한 번은 알려 줘야 합니다. CLAUDE.md 에 적어 두면 영구적으로 적용됩니다.
- Claude 의 추정을 그대로 믿는다. Claude 는 자신감 있게 틀릴 수 있습니다. 중요한 가정은 한 번 더 확인하세요.

Chapter 1.5

agent loop 이해하기

작업을 던지면 Claude 는 내부적으로 다음 다섯 단계를 반복합니다.

- 사용자 요청을 이해합니다. 모호하면 명확화 질문을 하나만 던집니다.
- 계획을 세웁니다. 큰 작업이면 todo 리스트로 분할해 사용자에게 먼저 보여줍니다.
- 도구를 호출합니다. 파일 읽기, 검색, 편집, 셀 실행, 웹 fetch 등.
- 결과를 합성합니다. 도구 출력을 읽고 다음 단계를 결정합니다.
- 결과를 보고하거나 다시 1단계로 돌아갑니다.

이 다섯 단계 루프를 agent loop 이라고 부르며, Claude Code 가 다른 단발 챗봇과 가장 다른 점입니다.

!Claude Code 의 agent loop — 한 번 응답하기까지 5 단계를 5~50 회 반복합니다 사용자는 "한 마디 → 한 답" 이 아니라 "한 마디 → 여러 도구 호출 → 한 종합 결과" 를 받게 됩니다.

도구의 종류

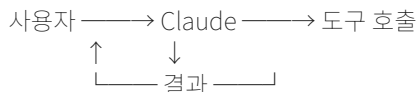
Claude 가 호출할 수 있는 도구는 다음 카테고리로 나뉩니다.

- 파일 도구: Read, Edit, Write, Glob (파일 목록), Grep (검색)
- 셸 도구: Bash (Linux/macOS), PowerShell (Windows)
- 웹 도구: WebFetch, WebSearch

- 계획 도구: TodoWrite (todo 리스트 관리)
- MCP 도구: 외부 서비스가 등록한 도구 (GitHub, Slack, Linear 등)
- Computer use: 데스크톱 앱·웹 페이지 제어

매 호출마다 Claude 는 어떤 도구가 필요한지 스스로 판단합니다. 사용자가 직접 도구 선택을 강제할 필요는 없지만, "Read 도구로 이 파일 좀 봐 줘" 같은 명시적 요청도 가능합니다.

호출 흐름의 시각적 이해



이 화살표 사이클을 한 번 도는 데 보통 1~3초 걸리고, 한 작업 단위에서 5~50회 반복됩니다. 그래서 작업 끝에 사용자가 받는 메시지가 "방금 5분 동안 Claude 는 23 번 도구를 호출했고..." 같은 모양이 됩니다.

Chapter 1.6

CLAUDE.md 첫 작성

CLAUDE.md 는 리포지토리 루트에 두는 한 페이지 분량의 프로젝트 안내문입니다. Claude Code 는 모든 세션 시작 시 이 파일을 자동으로 읽어 컨텍스트로 사용합니다. 다른 사람의 코드 베이스를 이해할 때 사용자가 가장 먼저 읽는 README 와 같은 역할이라고 보면 됩니다.

/init 으로 자동 생성

첫 작성은 명령 한 줄로 끝낼 수 있습니다.

```
> /init
```

Claude 가 디렉터리 구조와 package.json 을 자기가 읽어 초안을 만들어 줍니다. 그 초안을 손보는 게 사람의 일입니다.

좋은 CLAUDE.md 의 구성 요소

프로젝트 한 줄 설명

이 서비스가 무엇이고 누구를 위한 것인가.

디렉터리 구조

- `apps/web/` — Next.js 프론트엔드

- `apps/api/` — Fastify 백엔드

- `packages/db/` — Prisma 모델 + 마이그레이션

자주 쓰는 명령

- `pnpm dev` — 전체 개발 서버 띄우기
 - `pnpm test` — 단위 테스트
 - `pnpm e2e` — Playwright e2e
 - `pnpm lint` — ESLint + Prettier

컨벤션

- 모든 PR 은 한 문장 영문 제목 + 본문 한국어로 작성한다.
 - 함수 이름은 동사로 시작.
 - 데이터베이스 마이그레이션은 별도 PR 로 분리.
 - 외부 API 호출은 항상 zod 로 응답을 검증한다.

절대 건드리면 안 되는 것

- `packages/legacy-billing/` — 외부 PG 인증 통과 코드. 변경 시 재심사 필요.
 - `infra/terraform/prod/` — 프로덕션 인프라. 별도 승인 필요.

CLAUDE.md 는 길게 쓰지 마세요. 한 페이지가 넘어가면 Claude 는 마지막 절반을 자주 잊습니다. 진짜 자주 참조해야 하는 정보 — 명령, 컨벤션, 절대 건드리면 안 되는 것 — 만 짧게 적어 두는 게 좋습니다.

글로벌 가이드

~/ .claude/CLAUDE.md 에 쓴 내용은 모든 프로젝트에 공통으로 적용됩니다. "응답은 한국어로", "코드 변경 후 항상 테스트 실행" 같은 본인 작업 스타일을 적어 두면 매번 반복할 필요가 없습니다.

프로젝트 가이드 vs 글로벌 가이드 우선순위

두 파일이 동시에 있을 때 우선순위는 다음과 같습니다 (위가 우선):

- 프로젝트의 **./CLAUDE.md**
- 사용자 글로벌 **~/ .claude/CLAUDE.md**
- 머신 정책 (관리자 강제, Part 7 참조)

서로 충돌하면 Claude 는 위 순서대로 우선 적용합니다. 충돌하지 않는 항목은 둘 다 활용됩니다.

Chapter 1.7

흔한 막힘과 해결 (초급)

처음 Claude Code 를 쓸 때 자주 마주치는 7 가지 막힘과 해결법입니다.

1. "응답이 영어로 나옵니다"

~/**.claude/CLAUDE.md** 에 다음 한 줄을 추가하세요.

- 사용자와의 대화는 항상 한국어로 한다. 코드 주석·커밋 메시지는 영어 우선.

2. "Claude 가 자꾸 권한을 묻습니다"

자주 쓰는 안전한 명령은 자동 허용으로 등록할 수 있습니다.

> /permissions

여기서 "Always allow" 로 바꿀 수 있는 것: **git status, git diff, git log, npm test, pnpm test, cargo build**, 읽기 전용 SQL 등. 모든 쓰기·삭제·푸시 명령은 매번 묻도록 두는 게 안전합니다.

3. "응답이 너무 길어 화면이 넘칩니다"

> /compact

대화 컨텍스트를 요약해 토큰을 절약합니다. 토큰 절약뿐 아니라 응답 속도도 빨라집니다.

4. "Claude 가 같은 내용을 반복 설명합니다"

이전 응답에서 이미 설명한 내용을 다시 길게 설명하는 경우가 있습니다. 다음으로 멈출 수 있습니다.

> 이걸 이미 알고 있어. 결론만 한 줄로.

5. "코드 변경이 너무 큰 범위로 들어옵니다"

본인이 부탁한 변경 외에 코드 스타일·import 정렬·주석 정리 같은 무관한 변경이 끼어 들어가는 경우. CLAUDE.md 에 다음을 추가하세요.

- 변경은 요청 범위만. 코드 스타일·import 정리 같은 무관한 변경 금지.
- 의도하지 않은 추가 변경이 필요하면 먼저 물어볼 것.

6. "갑자기 인증이 풀렸습니다"

> /login

토큰은 보통 30~90 일 단위로 만료됩니다. 회사 계정의 경우 IT 정책에 따라 더 짧을 수 있습니다.

7. "다른 컴퓨터에서 같은 세션을 이어가고 싶습니다"

세션은 자동으로 클라우드에 동기화됩니다. 다른 기기에서 다음 명령으로 이어갈 수 있습니다.

```
claude --continue      # 가장 최근 세션
claude --resume <session-id> # 특정 세션
claude --list-sessions  # 사용 가능한 세션 목록
```

PART 2

일상 사용 (초급→중급)

Part 1 을 끝낸 사람은 이미 Claude Code 의 기본기를 손에 쥔 상태입니다. Part 2 는 매일 쓰면서 자연스럽게 마주치는 기능 8 가지를 깊이 다룹니다. 이걸 익히고 나면 90% 의 작업을 Claude Code 안에서 끝낼 수 있습니다.

Chapter 2.1

CLAUDE.md 깊이 있게

Part 1 에서 다룬 기본 작성법을 넘어, 큰 프로젝트에서 자주 쓰이는 패턴들을 정리합니다.

모노레포에서의 다층 CLAUDE.md

큰 모노레포는 디렉터리마다 컨벤션이 다를 수 있습니다. Claude Code 는 디렉터리별 **CLAUDE.md** 를 인식합니다.

```
my-monorepo/
├── CLAUDE.md          # 모노레포 전체 규칙
├── apps/
│   ├── web/CLAUDE.md  # 프론트엔드 전용 규칙
│   └── api/CLAUDE.md  # 백엔드 전용 규칙
└── packages/
    └── db/CLAUDE.md    # DB 전용 규칙
```

Claude 가 작업하는 파일 경로에서 가장 가까운 CLAUDE.md 부터 차례로 읽고, 상위 CLAUDE.md 와 합쳐 컨텍스트로 사용합니다.

복잡한 컨벤션 인코딩 패턴

좋은 CLAUDE.md 는 "왜" 까지 적습니다. "이렇게 하라" 만 적힌 규칙은 Claude 가 가끔 무시하지만, "왜냐하면 ..." 이 붙으면 충돌 시 우회 판단을 더 잘 합니다.

나쁜 예:

- 모든 함수는 async 로 작성.

좋은 예:

- 모든 외부 I/O 함수는 async. 동기 I/O 는 큰 트래픽에서 이벤트 루프를 막아 최근 인시던트(2026-02 결제 장애)의 원인이 되었음.

동적으로 생성되는 CLAUDE.md

CLAUDE.md 자체를 빌드 산출물로 두는 패턴도 가능합니다. **scripts/gen-claude-md.ts** 가 OpenAPI 스펙·DB 스키마·환경 변수 목록을 읽어 CLAUDE.md 를 자동 생성하면, Claude 가 항상 최신 메타 데이터로 작업할 수 있습니다.

CLAUDE.md 안 쓰면 어떻게 되나

물론 안 써도 동작합니다. 다만 Claude 가 매번 "이 프로젝트의 테스트 명령은 무엇인가요?" 같은 추정·질문을 반복하게 됩니다. 한 번 5분 들여 쓰면, 그 뒤로 평생 그 5분을 절약합니다.

Chapter 2.2

메모리 시스템 — # 와 /memory

Claude 는 세션이 끝나면 컨텍스트가 사라집니다. 다음 세션에 또 같은 설명을 반복하기 싫다면 메모리 를 활용하세요.

> # 이 프로젝트의 데이터베이스는 PostgreSQL 16 이고, 테스트에서는 Testcontainers 로 띄운다.

한 글자로 시작한 줄은 메모리로 저장됩니다. 다음 세션에 자동으로 다시 주입되어 같은 설명을 반복하지 않아도 됩니다.

저장된 메모리 보기·정리:

> /memory

메모리의 두 종류

- Project memory (`./.claude/memory/`) — 이 리포에서만 유효. 팀과 공유.
- User memory (`~/.claude/memory/`) — 모든 프로젝트에 유효. 본인만 사용.

저장할 때 어느 쪽에 넣을지 Claude 가 묻거나 합리적으로 추정합니다. 원치 않으면 직접 폴더로 옮기면 됩니다.

메모리 파일 구조

각 메모리는 마크다운 파일로 저장됩니다.

```
---
name: db_migration_protocol
description: 데이터베이스 마이그레이션 시 따라야 할 표준 절차
type: project
---
```

데이터베이스 마이그레이션 적용 순서:

1. 새 컬럼은 NULL 허용으로 추가
2. 백필 마이그레이션 별도 PR
3. NOT NULL 제약 추가는 백필 검증 후

type 필드는 **user**, **feedback**, **project**, **reference** 중 하나입니다.

- **user**: 사용자 본인의 역할·선호

- **feedback**: 사용자가 준 교정·확정 사항
- **project**: 진행 중인 작업·이슈·인시던트
- **reference**: 외부 시스템에 있는 정보의 위치

무엇을 메모리에 저장할 것인가

좋은 메모리는 "코드를 보면 자명하지 않은 사실"입니다.

- ☑ "이 프로젝트의 인증 토큰은 1시간 만료. 갱신은 cron 이 처리."
- ☑ "결제 모듈은 절대 수정 금지. 외부 PG 인증 통과 코드라 변경 시 재심사 필요."
- ☑ "팀 PR 표기는 conventional commits + 한국어 본문."
- ☑ "이 함수는 user 를 받아 string 을 반환한다." (코드를 읽으면 자명)
- ☑ "팀 슬랙 채널은 #engineering." (외부 시스템 위치는 reference 메모리로)

메모리 잊기

작업 진행 상태가 바뀌어 옛 메모리가 거짓이 되면 다음으로 정리합니다.

```
> /memory delete db_migration_protocol
```

또는 **/memory** 의 인터랙티브 화면에서 직접 편집·삭제할 수 있습니다. 오래된 메모리를 그대로 두면 Claude 가 가끔 옛날 정보로 작업하는 사고가 납니다.

Chapter 2.3

슬래시 명령

/ 로 시작하는 짧은 명령으로 자주 쓰는 동작을 빠르게 호출할 수 있습니다.

내장 슬래시 명령 (자주 쓰는 것)

- **/help** — 사용 가능한 명령 목록
- **/clear** — 현재 세션 컨텍스트 비우기
- **/compact** — 컨텍스트를 요약해 토큰 절약
- **/memory** — 메모리 관리
- **/cost** — 이번 세션 누적 토큰·비용
- **/login, /logout** — 계정 전환
- **/model** — 사용 모델 변경 (Sonnet, Opus, Haiku)
- **/init** — 현재 폴더에 **CLAUDE.md** 자동 생성

- `/permissions` — 권한 정책 관리
- `/config` — 설정 보기·변경 (2026-04 부터 `~/.claude/settings.json` 에 영구 저장)
- `/agents` — 사용 가능한 서브에이전트 목록
- `/mcp` — 등록된 MCP 서버 보기·토글
- `/checkpoint`, `/restore` — 체크포인트 저장·복원
- `/powerup` — 인터랙티브 튜토리얼 (2026-03 도입)
- `/ultraplan` — 클라우드에서 큰 계획 세우기 (2026-04 도입)
- `/autofix-pr` — 터미널에서 PR 자동 수정 (2026-04 도입)
- `/team-onboarding` — 본인 설정을 패키지로 (2026-04 도입)
- `/loop` — 반복 작업 (Monitor 도구와 결합 시 자가 페이싱, 2026-04)

사용자 정의 슬래시 명령

`./.claude/commands/<name>.md` 또는 `~/.claude/commands/<name>.md` 파일을 만들면 새 명령이 자동 등록됩니다.

```
---
description: PR 본문 초안 작성
argument-hint: <pr-number>
allowed-tools: Read, Bash(git log:*), Bash(git diff:*)
---
```

마지막 커밋 5개를 분석해 PR 본문을 작성해 주세요.

- 한국어 제목 (한 문장, 명령어형)
- 본문은 ## 변경 사항 / ## 검증 / ## 영향 받는 부분 세 섹션
- 자동 생성된 변경(빌드 산출물)은 제외

저장 후 `/draft-pr` 처럼 호출할 수 있습니다. 팀 공통 명령은 프로젝트의 `.claude/commands/` 에 두고 git 으로 공유하면 됩니다.

인자 받는 슬래시 명령

명령에 인자를 받고 싶다면 `$ARGUMENTS` 변수를 본문에서 사용합니다.

```
---
description: 이슈 번호로 PR 만들기
argument-hint: <issue-number>
---
```

이슈 `#$ARGUMENTS` 를 읽고, 그 이슈를 해결하는 변경을 만들어 주세요.

- 브랜치 이름: `fix/issue-$ARGUMENTS-<요약>`
- 커밋 메시지에 "Fixes `#$ARGUMENTS`" 포함

호출:

> /create-fix-pr 142

자주 만드는 사용자 정의 명령 5 종

조직에서 자주 만드는 명령 패턴입니다.

- **/draft-pr** — 변경 내역 요약해 PR 본문 초안
- **/explain-bug** — 스택 트레이스 받아 원인·해결 후보 설명
- **/coverage-report** — 테스트 커버리지 측정·미달 영역 보고
- **/db-migrate-check** — 마이그레이션 안전성 검토
- **/release-notes** — 두 태그 사이 변경 정리해 릴리스 노트 작성

이런 명령을 5분 들여 만들어 두면, 같은 작업을 한 글자로 호출하게 되어 한 달에 수 시간을 절약합니다.

Chapter 2.4

파일 편집 워크플로

Claude Code 가 코드 편집할 때 따르는 패턴:

- 읽기 먼저. 편집 전에 항상 해당 파일을 다시 읽어 현재 상태를 확인합니다.
- 최소 변경. 의도에 필요한 부분만 손댁니다. 무관한 코드 스타일 변경을 끼워 넣지 않습니다.
- 확인 — 출력. 적용 후 결과 diff 를 사용자에게 보여 줍니다.

이 패턴을 신뢰할 수 있는 이유는 Claude Code 가 **Edit** 같은 정밀한 도구를 사용하기 때문입니다. **old_string** 과 **new_string** 을 정확히 맞추지 않으면 Edit 이 실패합니다. 그래서 "이상한 부분까지 같이 바뀌었네" 같은 사고가 적습니다.

큰 변경은 Plan 모드를 활용하라

여러 파일에 걸친 리팩터링은 변경을 시작하기 전에 계획을 받아 보는 게 좋습니다.

> 먼저 계획부터 세워 주세요. apps/api/src/services 에 있는 모든 svc 클래스를 함수형으로 리팩터링하고 싶습니다.

Claude 는 (1) 어떤 파일을 만질지, (2) 어느 순서로 만질지, (3) 어떤 위험이 있는지를 먼저 보여 줍니다. 마음에 들지 않는 부분은 그 자리에서 수정 요청하고, 동의하면 "그대로 진행" 한 마디로 시작시킵니다.

Plan 모드 토글

Shift+Tab 으로 plan 모드 ↔ 일반 모드를 토글할 수 있습니다. plan 모드에서는 Claude 가 절대 파일을 변경하지 않고 계획만 세웁니다.

큰 리팩터링 자르기

전체 모듈 리팩터링이 너무 크다면 다음처럼 자르세요.

- 첫 단계: 한 파일만 리팩터링해 패턴을 잡는다.
- 두 번째 단계: 같은 패턴을 다른 파일에 적용한다 (Claude 가 자동으로 반복 가능).
- 세 번째 단계: 통합 테스트 · e2e 까지 통과시킨다.

각 단계 끝에 commit 을 따로 두면, 어디서 사고가 났을 때 이전 단계로 git revert 하기 쉽습니다.

패턴 학습 시키기

비슷한 작업을 여러 번 시킬 때는, 첫 번째 결과를 보고 패턴을 명시적으로 알려 주세요.

> 좋아, 이 함수 변환 결과 마음에 들어. 같은 패턴으로 svc-user.ts, svc-order.ts, svc-billing.ts 도 변환해 줘.

이렇게 하면 Claude 는 첫 번째 결과를 템플릿으로 사용해 나머지를 일관되게 처리합니다.

Chapter 2.5

터미널 통합

Claude Code 는 셸과 한몸처럼 동작하도록 설계됐습니다. 다음 패턴을 알아 두면 효율이 크게 옵니다.

비대화형 모드 (-p 또는 --print)

스크립트나 CI 에서 Claude 를 한 번 호출하고 싶다면:

```
claude -p "이 디렉터리 안의 .env.sample 을 모든 .env 에 적용해 누락 키를 채워 달라" \  
  --output-format json \  
  > result.json
```

-p 는 단발 프롬프트, **--output-format** 으로 JSON 으로 받으면 후처리하기 편합니다.

2026-04 부터는 **--print** 모드도 에이전트의 **tools:** / **disallowedTools:** 프론트매터를 따라 인터랙티브 모드와 동일하게 동작합니다.

파이프 입력

표준 입력으로 데이터를 흘려 넣을 수도 있습니다.

```
git diff main | claude -p "이 PR 의 위험 요소 5개를 표로 정리해 주세요"
```

이 패턴은 PR 검토, 로그 분석, 에러 메시지 해석 같은 단발 작업에 매우 잘 맞습니다.

--from-pr 로 PR 직접 작업

2026-04 부터 GitHub PR URL 뿐 아니라 GitLab MR, Bitbucket PR, GitHub Enterprise PR 모두 지원합니다.

```
claude --from-pr https://github.com/owner/repo/pull/123
```

PR 의 변경 사항을 자동으로 fetch 하고 그 위에서 검토·수정·재푸시까지 한 번에 진행할 수 있습니다.

셸 사용에 동의 한 번 받기

Claude 가 처음으로 위험할 수 있는 명령 — 파일 삭제, 강제 푸시, 관리자 권한 — 을 시도하면 사용자에게 동의를 묻습니다. 매번 묻지 않게 하려면 **--allow** 옵션이나 슬래시 명령 **/permissions** 로 세션 정책을 조정할 수 있습니다. 보안 측면은 Part 5 에서 자세히 다룹니다.

Auto mode (2026-03 도입)

Shift+Tab 으로 cycle 가능. Auto mode 에서는:

- 안전한 변경 (편집·읽기·테스트 실행) 은 자동 통과
- 위험한 변경 (파일 삭제·푸시·외부 API 호출) 은 차단·확인 요청
- 매번 묻는 일반 모드와 **--dangerously-skip-permissions** 의 중간 지점

영구 기본값으로 두려면 **.claude/settings.json** 에:

```
{
  "permissions": {
    "defaultMode": "auto"
  }
}
```

Headless 환경 (CI 등)

CI 에서 Claude 를 자동 호출할 때는 인증 토큰을 환경 변수로 주입합니다.

```
# .github/workflows/claude-review.yml
- name: Claude review
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

```
run: |
  claude -p "${{ github.event.pull_request.body }}" \
    --tools-allow Read,Grep \
    --output-format json > review.json
```

이렇게 권한을 좁게 주면 CI 환경에서도 안전합니다.

Chapter 2.6

Plan 모드 깊이 있게

Plan 모드는 단순히 "변경 안 하고 계획만" 이 아니라, 큰 작업을 안전하게 끝내는 핵심 도구입니다.

언제 Plan 모드를 써야 하는가

다음 중 하나라도 해당되면 Plan 모드부터 시작하세요.

- 5 개 이상 파일이 변경될 가능성이 있다.
- 데이터베이스 · 인증 · 결제 같은 위험 모듈을 만진다.
- 테스트가 충분치 않은 영역의 리팩터링.
- 처음 다루는 코드 베이스에서의 큰 변경.
- 마이그레이션 · 배포 등 되돌리기 어려운 작업.

Plan 검토 체크리스트

Claude 의 계획을 받았을 때 사용자가 확인할 5 가지:

- 목표 일치: 내가 부탁한 것과 정확히 일치하는가?
- 범위 적절: 너무 크거나 작지 않은가? 자를 수 있는가?
- 순서 안전: 먼저 해야 할 것이 먼저 오는가? (예: 테스트 먼저, 변경 다음)
- 위험 식별: Claude 가 위험을 명시했는가? 명시되지 않은 위험이 있는가?
- 롤백: 망가지면 어디로 되돌릴 수 있는가?

이 5 가지가 충족되면 "그대로 진행" 으로 시작시킵니다. 충족되지 않으면 그 자리에서 수정 요청하고 다시 받습니다.

Ultraplan (2026-04 도입)

Plan 모드의 클라우드 버전입니다. 큰 계획을 작성하는 동안 본인 터미널은 자유. 클라우드의 Claude Code on the Web 세션이 계획을 그리고, 브라우저에서 검토 · 수정 · 승인.

```
> /ultraplan migrate the auth service from sessions to JWTs
```


승인 후 (1) 클라우드에서 직접 실행하거나 (2) 본인 CLI 로 다시 보내 로컬에서 실행할 수 있습니다.

Chapter 2.7

Git 통합

Claude Code 는 Git 과 일상적으로 협업합니다. 다음 패턴들이 가장 유용합니다.

의미 있는 커밋

> 변경 사항을 의미 있는 단위로 나눠 여러 커밋으로 만들어 주세요. 메시지는 conventional commits 형식.

Claude 는 **git diff** 를 읽어 변경을 논리 단위로 그룹화하고, 각 그룹을 별도 커밋으로 만듭니다.

PR 본문 초안

> origin/main 과의 차이를 보고 PR 본문 초안을 만들어 주세요. 한국어, 세 섹션 (변경/검증/영향).

머지 컨플릭트 해결

> main 에서 rebase 하다가 컨플릭트가 났어요. apps/api/src/router.ts 의 충돌을 보고 의도를 살리는 방향으로 해결해 주세요.

Claude 는 양쪽 변경을 읽고 의도를 추정해 통합합니다. 자기가 한 결정을 사용자에게 한 번 보여 주므로 검토 후 채택할 수 있습니다.

옛 커밋 추적

> apps/api/src/auth/jwt.ts 가 마지막으로 의미 있게 바뀐 게 언제인지, 그리고 그 변경이 무엇이었는지 요약.

git log -p --follow 와 **git blame** 을 적절히 조합해 답변합니다.

Git 위에서의 위험 동작

Claude 가 자동으로 하지 못하게 막아 두는 게 좋은 동작들 (또는 매번 확인):

- **git push --force** (특히 **main/master** 브랜치)
- **git rebase** (이미 push 된 브랜치)
- **git reset --hard**
- **git branch -D**
- **git tag -d**

- 다른 사용자의 커밋을 덮어쓰는 동작

/permissions 로 이 명령들을 "Always ask" 로 두고 시작하세요.

Chapter 2.8

권한 설정 첫발

권한 정책은 Part 5 에서

PART 3

도구 확장 (중급)

여기부터가 본격 재미입니다. Part 1·2 가 "기본 차로 출퇴근하기" 였다면, Part 3 는 "차에 트렁크·루프랙·견인고리를 다는 단계" 입니다. 똑같이 한 시간 작업하는데, 도구를 잘 갖춘 사람은 결과물의 양이 3~5배 차이 납니다.

다섯 가지를 차례로 다룹니다 — MCP, Skills, Plugins, Hooks, Subagents. 한 번에 다 익힐 필요는 없어요. MCP 부터 시작해서 본인 워크플로에 맞는 것 하나씩 추가하면 충분합니다.

Chapter 3.1

MCP — 외부 도구 연결 (개요)

MCP (Model Context Protocol) 는 Claude Code 에 외부 서비스를 한 단계로 연결하는 표준 인터페이스입니다. 직관적으로는 "Claude 가 사용할 수 있는 USB 포트의 표준" 이라고 보면 됩니다. USB 가 키보드·마우스·외장하드를 한 포트로 다 받듯이, MCP 도 GitHub·Slack·DB·내부 API 를 한 인터페이스로 다 받습니다.

왜 MCP 가 중요한가

MCP 없이 Claude Code 를 쓰면, "이슈 #142 의 댓글 좀 가져와 줘" 같은 단순 요청도 사용자가 매번 GitHub URL 을 열어 본문을 복사해 붙여 넣어야 합니다. MCP 가 있으면 Claude 가 직접 GitHub API 를 호출해 가져옵니다. 이 한 단계 차이가 일상의 마찰을 크게 줄입니다.

추가는 한 줄

MCP 서버 등록은 한 줄 명령으로 끝납니다.

```
claude mcp add github -- npx -y @modelcontextprotocol/server-github
```

설치 후 다음 세션부터 Claude 가 GitHub 의 이슈·PR 도구를 자기가 알아서 호출합니다. 처음 호출 시 한 번 OAuth 인증을 거치고, 그 뒤부터는 자동.

등록한 서버 보기

```
claude mcp list
```

또는 세션 안에서:

```
> /mcp
```

토글·제거

특정 MCP 를 일시적으로 끄고 싶다면 **/mcp** 의 인터랙티브 화면에서 토글. 영구 제거는:

```
claude mcp remove github
```

Chapter 3.2

자주 쓰는 MCP 톱 10

설치할지 말지 고민될 때 참고할 추천 목록입니다. 본인 워크플로에 한 번이라도 등장하는 서비스가 있으면 일단 깔아 두고, 안 쓰면 빼는 식으로 다듬으세요.

- GitHub MCP — 이슈·PR·코드 검색·머지. 가장 처음 깔 만한 것. `npx -y @modelcontextprotocol/server-github`
- Slack MCP — 채널 검색·메시지 전송. "어제 #engineering 에서 결제 관련 논의 요약해 줘" 같은 게 됩니다.
- Linear MCP — 티켓 이동, 스프린트 진행 상태. 회사 워크플로가 Linear 면 필수.
- Notion MCP — 페이지·DB 쿼리. 회사 위키가 Notion 이면 매우 유용.
- Postgres MCP — 안전한 SELECT 쿼리. 운영 DB 에 직접 붙는 위험 때문에 꼭 read-only 모드로 시작.
- Filesystem MCP — 작업 폴더 외부의 안전한 디렉터리 접근. 다른 모노레포·다른 프로젝트도 같이 만질 때.
- Brave Search / Tavily MCP — 외부 웹 검색. 공식 docs 보강·뉴스 수집.
- Puppeteer MCP — 헤드리스 브라우저. 페이지 렌더링·스크린샷·동작 검증.
- Sentry MCP — 에러 트래커. "최근 24h 의 에러 톱 10 과 영향 받은 사용자 수" 같은 질의.
- PagerDuty / Opsgenie MCP — 인시던트 관리. 온콜·에스컬레이션.

설치는 모두 비슷합니다. 공식 [MCP 마켓플레이스](#) 에서 한 번에 검색할 수 있습니다.

깔고 후회한 패턴

너무 많이 깔면 응답이 느려집니다. Claude 는 매 호출마다 사용 가능한 도구 목록을 검토하므로, 도구가 100개 등록되어 있으면 그만큼 토큰·시간이 듭니다. "이번 주에 한 번도 안 쓴 MCP 는 빼자" 정도의 가벼운 룰을 두면 좋습니다.

Chapter 3.3

사용자 정의 MCP 서버 만들기

타인이 만든 MCP 로 만족 못 하면, 직접 만들 수 있습니다. 회사 내부 API 를 Claude 가 직접 부르게 하는 게 가장 흔한 동기.

가장 짧은 MCP 서버 (Python)

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("my-internal-api")

@mcp.tool()
def get_customer(customer_id: str) -> dict:
    """우리 회사 고객 정보 조회. customer_id 로 조회한다."""
```

```
# 실제 내부 API 호출
import requests
r = requests.get(f"https://api.internal/customers/{customer_id}",
                 headers={"Authorization": f"Bearer {os.environ['INTERNAL_TOKEN']}"})
return r.json()

if __name__ == "__main__":
    mcp.run()
```

Claude Code 에 등록

```
claude mcp add my-api -- python /path/to/my_mcp_server.py
```

이걸로 Claude 가 "고객 12345 의 결제 상태 좀 봐 줘" 같은 자연어를 받으면, 자동으로 **get_customer("12345")** 를 호출합니다.

만들 때 신경 써야 하는 것

- 함수 docstring 을 정성껏 쓰기. Claude 는 docstring 만 보고 도구를 언제 쓸지 결정합니다. "고객 정보 조회" 보다 "회사 고객 ID 로 결제·구독·연락처 정보를 조회" 가 훨씬 잘 트리거됩니다.
- 반환값을 사람이 읽을 수 있게. dict 의 키 이름을 **cust_id** 보다 **customer_id** 처럼 명료하게.
- 에러 메시지가 친절해야 함. "404" 보다 "Customer not found: 12345. 정확한 ID 인지 확인해 주세요." 가 Claude 가 다음 단계를 결정하는 데 도움이 됩니다.
- 읽기/쓰기 분리. 조회 도구와 변경 도구를 분리하면, 변경 도구만 권한 정책으로 막을 수 있습니다.

Chapter 3.4

Skills — 특정 작업 모범 사례

Skill 은 "이 작업은 이렇게 처리해라" 라는 모범 사례·가이드 묶음입니다. SKILL.md 파일에 트리거 조건과 절차를 적어 두면, 해당 패턴이 등장할 때 Claude 가 자동으로 그 가이드를 따릅니다.

기본 제공되는 Skills 일부:

- pdf — PDF 읽기·생성·병합·폼 작성
- docx / pptx / xlsx — 오피스 파일 처리
- schedule — 반복 일정 작업 만들기
- skill-creator — 새 스킬 만드는 도우미
- engineering:documentation — 기술 문서 (README·런북·API)
- engineering:code-review — 보안·성능·정확성 리뷰

스킬은 마켓플레이스에서 패키지 단위로 받거나 직접 만들 수 있습니다.

Chapter 3.5

Skills 만들기 (실전)

직접 만드는 게 의외로 쉽습니다. 한 번 만들어 두면 팀 전체가 같은 절차를 따르게 되어 큰 가치가 생깁니다.

디렉터리 구조

```
mkdir -p ~/.claude/skills/my-pr-flow
```

```
~/.claude/skills/my-pr-flow/
├── SKILL.md      # 스킬 본체
├── reference.md  # (선택) 참조 문서
├── examples/     # (선택) 예시
│   └── good-pr.md
```

SKILL.md 예시

```
---
name: my-pr-flow
description: 우리 팀 PR 작성 절차. "PR 만들어 줘", "/pr" 또는 사용자가 PR 본문을 부탁할 때 사용.
---
```

```
# 우리 팀 PR 절차
```

```
## 트리거 조건
```

다음 중 하나에 해당할 때 이 스킬을 적용:

- 사용자가 "PR 만들어" / "PR 본문 작성" 같은 요청
- `/draft-pr` 같은 슬래시 명령
- `git push` 직후 자연스럽게 이어지는 흐름

```
## 절차
```

1. main 에서 분기한 브랜치인지 확인 (`git rev-parse --abbrev-ref HEAD`)
2. 변경 파일이 5개 이상이면 사용자에게 한 번 의견을 묻고 진행
3. 본문은 다음 세 섹션:
 - ## 변경 사항 (왜)
 - ## 검증 방법 (어떻게 확인했는지)
 - ## 위험 (롤백 시점)
4. 한 줄 영문 제목 (명령형, 50자 이내), 본문은 한국어
5. 자동 생성 변경 (build/dist) 은 제외

```
## 좋은 예 / 나쁜 예
```

자세한 예시는 examples/good-pr.md 참고.

저장 후부터 "PR 본문 부탁" 같은 요청에 Claude 가 위 절차를 자동 적용합니다.

description 잘 쓰기

스킬은 description 의 키워드로 트리거됩니다. 모호하면 안 트리거되고, 너무 광범위하면 엉뚱한 데서 트리거됩니다. 좋은 예:

- ❌ "PR 만들기" — 너무 광범위
- ❌ "git workflow" — 영문만이라 한국어 트리거 못 잡음
- ❌ "우리 팀 PR 작성 절차. 'PR 만들어 줘', '/pr', 'pull request' 등으로 트리거"

스킬 vs 슬래시 명령

비슷해 보이지만 차이가 있습니다.

- 슬래시 명령: 사용자가 `/`으로 명시적 호출. 짧은 단발 작업.
- 스킬: Claude 가 자연어 맥락에서 자동 적용. 긴 절차·정책 인코딩.

같은 동작을 둘 다 만들 수도 있습니다. `/draft-pr` 슬래시 명령이 my-pr-flow 스킬을 호출하는 식.

Chapter 3.6

Plugins — 묶음 설치

Plugin 은 Skills · MCPs · 슬래시 명령을 한 묶음으로 패키징한 단위입니다. 본인이나 팀의 작업 묶음을 한 번에 배포할 때 사용합니다.

마켓플레이스에서 설치

```
claude plugin install engineering@knowledge-work-plugins
```

이 한 줄로 engineering 플러그인의 10개 스킬 (architecture, code-review, debug, deploy-checklist, documentation, incident-response, standup, system-design, tech-debt, testing-strategy) 이 한 번에 추가됩니다.

마켓플레이스 둘러보기:

```
claude plugin search "documentation"
claude plugin search "design"
claude plugin info engineering@knowledge-work-plugins
```

자주 쓰는 마켓플레이스 플러그인

- engineering — 기술 문서, 리뷰, 디버그, 배포 체크리스트

- finance — 회계 마감, SOX 테스트, 재무제표
- operations — 프로세스 문서, 런북, 변경 요청
- human-resources — 채용 · 온보딩 · 평가
- design — 디자인 시스템 감사, 핸드오프, 접근성
- brand-voice — 브랜드 톤 적용 · 가이드라인 생성

플러그인 토글

```
claude plugin disable engineering@knowledge-work-plugins
claude plugin enable engineering@knowledge-work-plugins
```

Chapter 3.7

플러그인 만들기 · 배포

본인 팀의 워크플로를 플러그인으로 묶으면 동료에게 한 줄 명령으로 배포할 수 있습니다.

디렉터리 구조

```
my-team-plugin/
├── plugin.json
├── README.md
├── skills/
│   ├── pr-flow/SKILL.md
│   ├── deploy/SKILL.md
│   └── postmortem/SKILL.md
├── commands/
│   ├── ship.md
│   └── rollback.md
├── mcp/
│   └── servers.json
```

plugin.json

```
{
  "name": "acme-engineering",
  "version": "1.0.0",
  "description": "Acme 엔지니어링 팀 워크플로 묶음",
  "author": "Acme Team",
  "skills": ["pr-flow", "deploy", "postmortem"],
  "commands": ["ship", "rollback"],
  "mcps": ["acme-internal-api"]
}
```

로컬에서 테스트

```
claude plugin install ./my-team-plugin
```

사내 마켓 또는 직접 배포

.plugin 확장자로 zip 해 동료에게 전달하거나 사내 마켓플레이스에 등록. Claude Code 의 **cowork-plugin-management:create-cowork-plugin** 스킬이 가이드를 단계별로 도와줍니다.

/team-onboarding 으로 본인 설정 패키징 (2026-04 도입)

플러그인을 만드는 게 부담스럽다면, 본인 현재 설정 (CLAUDE.md, 슬래시 명령, MCP 목록) 을 한 묶음으로 자동 패키징하는 명령이 있습니다.

```
> /team-onboarding
```

이 명령은 (1) 본인 설정을 분석해 (2) 새로 합류한 동료가 같은 환경을 받을 수 있는 plugin 을 만들어 줍니다. 신입 온보딩 시 매우 유용합니다.

Chapter 3.8

Hooks — 자동 실행 후크

Hook 은 특정 이벤트가 발생했을 때 자동으로 실행되는 사용자 정의 스크립트입니다.

다섯 가지 후크 타입

- PreToolUse — 도구 호출 직전. 위험한 명령을 가로채 검증·차단할 때.
- PostToolUse — 도구 호출 직후 (성공). 결과를 가공해 후처리.
- PostToolUseFailure — 도구 호출 실패 시. 알림·로깅.
- SessionStart — 세션 시작 시. 환경 점검·인사말.
- Stop — 응답 완료 시. 알림 보내기.

가장 흔한 사용 — 위험 명령 차단

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "if": "Bash(rm -rf /*)",
        "command": "block-rm.sh"
      },
      {
```

```

    "matcher": "Bash",
    "if": "Bash(git push --force*)",
    "command": "confirm-force-push.sh"
  }
]
}
}

```

if 조건에 맞는 도구 호출이 발생하면 외부 스크립트가 실행되고, 그 스크립트가 **exit 2**로 종료되면 도구 호출 자체가 중단됩니다.

또 다른 흔한 사용 — Slack 알림

```

{
  "hooks": {
    "Stop": [
      {
        "command": "send-slack-summary.sh"
      }
    ]
  }
}

```

세션이 끝날 때마다 본인 Slack 채널로 변경 요약을 보내게 만들 수 있습니다.

어디에 두는가

- 글로벌: `~/.claude/settings.json`의 **hooks** 섹션
- 프로젝트: `./.claude/settings.json`
- 관리자 강제: 회사 PC의 시스템 정책 위치 (Part 7 보안 챕터 참조)

혹은 코드를 줄이는 게 아니라 위험을 차단하고 정책을 강제하는 용도로 가장 강력합니다.

2026-04 추가 — duration_ms

PostToolUse와 **PostToolUseFailure** 후크 입력에 **duration_ms** (도구 실행 시간)가 포함됩니다. 느린 도구를 찾는 데 유용합니다.

```

#!/bin/bash
# slow-tool-watch.sh
input=$(cat)
duration=$(echo "$input" | jq -r .duration_ms)
tool=$(echo "$input" | jq -r .tool_name)
if [ "$duration" -gt 30000 ]; then
  echo "⚠️ $tool 호출이 ${duration}ms 걸렸습니다." >&2
fi

```

Chapter 3.9

Subagents — 서브에이전트

Subagent 는 특정 역할에 특화된 보조 Claude 인스턴스입니다. "이 단계에서는 코드 리뷰만 해 줘" 같은 좁은 임무를 맡길 때 씁니다.

내장 서브에이전트

- code-reviewer — diff 를 읽고 보안·성능·정확성 관점으로 리뷰
- explorer — 코드 베이스 탐색·검색에 특화
- plan — 구현 전에 단계별 계획만 작성
- statusline-setup — Claude Code 상태 줄 커스텀
- general-purpose — 범용

직접 만들기

```
---
name: db-migrator
description: 데이터베이스 마이그레이션 전용. 스키마 변경 시 사용.
tools: Read, Edit, Bash
permissionMode: auto
---
```

당신은 PostgreSQL 마이그레이션 전문가입니다. 다음 규칙을 지킵니다:

1. NOT NULL 컬럼 추가 시 항상 백필 마이그레이션 분리
2. 50M 이상 테이블의 ALTER 는 CONCURRENTLY 옵션 우선
3. 모든 변경에 롤백 SQL 첨부
4. 마이그레이션 PR 본문에 다음 세 가지 명시: 다운타임 여부 / 인덱스 영향 / 롤백 시점

서브에이전트 호출

> 마이그레이션 부분은 db-migrator 에게 맡겨 주세요.

또는 Task 도구를 사용해 명시적 호출:

> Task 도구를 써서 db-migrator 서브에이전트에게 packages/db/migrations 검토를 부탁해 주세요.

서브에이전트의 가치

- 컨텍스트 분리: 메인 Claude 의 대화에 마이그레이션 세부가 안 끼어들어 깔끔.
- 권한 분리: 서브에이전트마다 다른 도구·권한 모드.
- 재사용: 같은 역할을 여러 프로젝트에서 호출.

서브에이전트는 Part 4 의 Agent SDK 와 결합하면 더 강력해집니다. 거기서 다시 다룹니다.

PART 4

Agent SDK (중급→고급)

여기서부터는 "Claude Code 를 도구로 쓰는" 단계를 넘어, *****Claude Code 자체를 부품으로 자기 어플리케이션·CI·서버 안에 끼우는***** 단계입니다. CLI 안에서가 아니라 본인 코드 안에서 Claude 의 agent loop 를 직접 돌립니다.

이 Part 는 다음 사람에게 꼭 필요합니다.

- 사내 도구·플랫폼에 AI 기능을 붙이려는 백엔드 개발자
- CI/CD 의 코드 리뷰·테스트 보조에 Claude 를 끼우려는 데브옵스
- 본인 SaaS 에 "AI 어시스턴트" 기능을 추가하려는 인디 해커

전체를 한 번에 읽지 마세요. SDK 개요와 첫 예제 두 챕터부터 시작해 본인 use case 에 맞는 챕터로 점프하면 됩니다.

Chapter 4.1

Agent SDK 개요

Claude Agent SDK 는 Claude Code 의 agent loop 를 자기 코드 안에서 직접 돌리게 해 주는 라이브러리입니다. 파이썬과 타입스크립트 두 가지 언어를 공식 지원합니다.

가장 짧은 사용 예 (Python):

```
from anthropic_agent import Agent

agent = Agent(
    model="claude-opus-4-6",
    system="당신은 우리 회사의 코드 리뷰어입니다.",
    tools=["read_file", "grep", "edit_file", "run_tests"],
)

result = agent.run("apps/api 디렉터리의 보안 문제 다섯 개를 찾아 표로 정리해 주세요")
print(result.final_message)
```

Agent SDK 는 다음을 직접 제어할 수 있게 해 줍니다.

- 시스템 프롬프트 — Claude 의 정체성·규칙을 자기 도메인에 맞게.
- 도구 집합 — 기본 도구 외에 자기 회사 API 도 도구로 등록.
- 승인 흐름 — 위험한 도구 호출 전에 사용자에게 확인받는 정책.
- 세션 상태 — 멀티턴 대화·체크포인트·메모리.

자세한 API 는 공식 문서의 [Agent SDK reference](#) 페이지를 참고하세요. 본 가이드북에서는 "왜 SDK 를 쓰는가" 와 "기본 패턴" 만 다룹니다.

CLI 와 SDK 의 차이

```
| 측면 | Claude Code CLI | Agent SDK |
| :--- | :--- | :--- |
| 실행 환경 | 사용자의 셸 | 본인 어플리케이션 코드 |
| 사용자 인터페이스 | 터미널 대화 | 본인이 만든 UI |
| 도구 집합 | 내장 + MCP | 본인이 정의 |
| 인증 | 사용자 토큰 | API 키 / Bedrock / Vertex |
| 사용 사례 | 일상 코딩 | 제품 기능 / 자동화 |
```

Chapter 4.2

Python SDK 빠른 시작

설치

```
pip install anthropic-agent
```

첫 예제

```
from anthropic_agent import Agent

agent = Agent(model="claude-sonnet-4-6")

# 단발 질문
result = agent.run("Python 의 GIL 을 한 문단으로 설명해 주세요")
print(result.final_message)

# 멀티턴
agent.run("우리 회사는 음식 배달 SaaS 야")
agent.run("우리 데이터를 분석할 때 가장 흥미로운 질문 5개를 추천해 줘")
```

도구 추가

```
from anthropic_agent import Agent, tool

@tool
def get_weather(city: str) -> dict:
    """주어진 도시의 현재 날씨를 조회한다."""
    import requests
    r = requests.get(f"https://wttr.in/{city}?format=j1")
    return r.json()

agent = Agent(
    model="claude-sonnet-4-6",
    tools=[get_weather],
)
agent.run("서울이랑 부산 중에 오늘 어디 날씨가 더 좋아?")
```

함수 docstring 을 잘 써야 한다는 원칙은 SDK 에서도 그대로입니다.

시스템 프롬프트로 정체성 부여

```
agent = Agent(
    model="claude-sonnet-4-6",
    system="""당신은 SONSLAB 블로그의 글쓰기 도우미입니다.

규칙:
- 답변은 항상 한국어 자연스러운 톤
- 영어 기술 용어는 그대로 두되, 일반 단어는 한국어로
- 출처 없는 주장은 하지 않는다
- 모르는 건 모른다고 말한다
""",
)
```

이 system 메시지는 모든 호출에서 유지됩니다.

Chapter 4.3

TypeScript SDK 빠른 시작

설치

```
npm install @anthropic/agent
```

첫 예제

```
import { Agent } from "@anthropic/agent";

const agent = new Agent({
  model: "claude-sonnet-4-6",
});

const result = await agent.run("Tell me a one-line dad joke about closures");
console.log(result.finalMessage);
```

도구 추가

```
import { Agent, tool } from "@anthropic/agent";
import { z } from "zod";

const getWeather = tool({
  name: "get_weather",
  description: "주어진 도시의 현재 날씨를 조회한다.",
  parameters: z.object({
    city: z.string(),
  }),
  execute: async ({ city }) => {
    const r = await fetch(`https://wttr.in/${city}?format=j1`);
    return await r.json();
  },
});

const agent = new Agent({
  model: "claude-sonnet-4-6",
  tools: [getWeather],
});
```

TypeScript 쪽은 zod 로 도구 인자 스키마를 명시하는 것이 권장됩니다.

TypeScript V2 인터페이스 (preview)

2026-04 부터 V2 인터페이스가 미리보기로 제공됩니다. 더 간결한 API:

```
import { agent } from "@anthropic/agent/v2";

const reviewer = agent({
```

```
system: "당신은 보안 검토자입니다",
}).withTools({ getWeather });
```

```
await reviewer.send("auth.ts 의 잠재적 위험 5가지");
```

V2 의 핵심 변화는 immutable / chainable 빌더와 더 강한 TypeScript 추론입니다.

Chapter 4.4

시스템 프롬프트 수정

기본 Claude Code 의 시스템 프롬프트는 일반 코딩 보조에 맞춰져 있습니다. SDK 에서는 이걸 자기 도메인에 맞게 바꿀 수 있습니다.

누구를 위한 도우미인가 명시

```
system="""당신은 우리 회사의 시니어 백엔드 엔지니어 동료입니다.
```

```
당신과 대화하는 사람은 주니어 ~ 미드 레벨 개발자입니다.
```

- 모르는 거 물어볼 때 부끄러워하지 않는다
- 자기 코드 자랑하고 싶을 때도 있다
- 실수에 민감하다

```
당신의 역할:
```

- 코드 리뷰 시 친절하지만 정확하게
- 의견은 의견이라고 분명히 (의견 vs 사실 구분)
- 모르는 건 모른다고

```
"""
```

도메인 지식 주입

```
system="""당신은 우리 음식 배달 SaaS 의 백엔드를 담당합니다.
```

```
핵심 도메인 용어:
```

- "라이더" = 배달 기사
- "픽업" = 음식을 식당에서 받는 단계
- "드롭" = 고객에게 전달
- "ETA" = 예상 도착 시간 (라이더 위치 + 트래픽 기반)

```
자주 호출되는 외부 API:
```

- Tmap (한국 내비게이션)
- KakaoMaps (지오코딩)
- KG이니시스 (결제)

```
"""
```

이렇게 도메인 컨텍스트를 시스템 프롬프트에 박아 두면, "ETA 가 자꾸 5분씩 어긋나는데" 같은 모호한 입력도 정확히 이해합니다.

안 좋은 시스템 프롬프트 패턴

- ☒ "당신은 도움이 되는 어시스턴트입니다." — 너무 광범위해 의미 없음
- ☒ 5000자 넘는 길이 — 후반부가 자주 무시됨
- ☒ 모순되는 규칙 — "친절해라" + "절대 사람을 칭찬하지 마라"
- ☒ 형식 명령만 가득 — 무엇을 하지 말라는 규칙만, 무엇을 하라는 비전 없음

좋은 시스템 프롬프트는 "이 도우미가 누구이고, 누구를 위해, 어떤 가치를 주는가" 를 한 페이지 안에 답합니다.

Chapter 4.5

사용자 정의 도구 추가

SDK의 핵심 가치는 회사 내부 도구를 Claude의 도구로 등록할 수 있다는 것입니다.

Python — @tool 데코레이터

```
from anthropic_agent import tool
```

```
@tool
```

```
def get_user_orders(user_id: str, days: int = 30) -> list:
```

```
    """우리 회사 사용자의 최근 주문 내역을 조회한다.
```

```
    Args:
```

```
        user_id: 사용자 ID (UUID)
```

```
        days: 조회 기간 (기본 30일)
```

```
    Returns:
```

```
        주문 내역 리스트. 각 항목에는 order_id, items, total, status, created_at 포함.
```

```
    """
```

```
    return db.query(
```

```
        "SELECT order_id, items, total, status, created_at "
```

```
        "FROM orders WHERE user_id = ? AND created_at > NOW() - INTERVAL ? DAY",
```

```
        [user_id, days],
```

```
    ).fetchall()
```

도구 호출 권한 통제

```
from anthropic_agent import Agent
```

```
agent = Agent(
```

```
    tools=[get_user_orders, refund_order],
```

```
    permissions={
```

```
        "get_user_orders": "auto",    # 자동 허용
```

```
        "refund_order": "ask",        # 매번 사용자 확인
```

```
    },
```

```
)
```

도구 안에서 다른 도구 호출

도구 안에서 다른 도구를 호출하는 것도 가능. 단, 너무 깊이 중첩되면 디버그가 어려우니 2 단계 이내로 제한 권장.

Chapter 4.6

Streaming Output

기본적으로 **agent.run()** 은 응답이 완성될 때까지 기다립니다. 챗봇 UI 처럼 토큰 단위로 흘러 보내려면 stream 인터페이스를 사용.

Python

```
async for chunk in agent.run_stream("...질문..."):
    if chunk.type == "text":
        print(chunk.text, end="", flush=True)
    elif chunk.type == "tool_use":
        print(f"\n[도구 호출: {chunk.tool_name}]")
    elif chunk.type == "tool_result":
        pass # 표시 안 함
```

TypeScript

```
for await (const chunk of agent.runStream("...질문...")) {
    if (chunk.type === "text") {
        process.stdout.write(chunk.text);
    }
}
```

스트리밍은 사용자 경험에 직접 영향을 줍니다. 답변이 완성되기까지 5초 기다리는 것보다, 첫 토큰이 0.3초 만에 나오기 시작하면 훨씬 빠르게 느껴집니다.

Chapter 4.7

Streaming Input

반대로 사용자 입력을 스트리밍으로 받는 것도 가능합니다. 음성 입력·라이브 스피치 등에서 유용.

```
from anthropic_agent import StreamingInput

inp = StreamingInput()

# 다른 곳에서 입력을 흘러 보냄
inp.append("이 PR 의 ")
inp.append("위험 요소를 ")
```

```
inp.append("3 개만 정리해 주세요")
inp.close()

result = agent.run(inp)
```

Chapter 4.8

Structured Output

응답을 자유 텍스트가 아니라 정해진 스키마 (JSON 등) 로 받고 싶다면.

```
from pydantic import BaseModel

class Risk(BaseModel):
    title: str
    severity: int # 1~5
    description: str

class RiskReport(BaseModel):
    risks: list[Risk]
    overall: str

result = agent.run(
    "이 PR 의 위험을 분석해 주세요",
    output_schema=RiskReport,
)
print(result.parsed.risks[0].title)
```

output_schema 를 지정하면 Claude 는 응답을 그 스키마에 맞춰 생성합니다. JSON 검증을 통과할 때까지 자체적으로 재시도합니다.

Chapter 4.9

Tool Search

도구가 100 개 이상이 되면 매 호출마다 모든 도구 정의를 컨텍스트에 넣는 게 부담입니다. Tool Search 는 작업에 필요한 도구만 동적으로 로드하는 최적화입니다.

```
agent = Agent(
    tools=load_all_internal_tools(), # 200개
    tool_search="auto", # 자동으로 관련 도구만 로드
)
```

내부적으로는 임베딩 기반 검색으로 입력 질문과 가장 관련 있는 톱 N 개 도구만 컨텍스트에 포함시킵니다. 토큰 절약 + 응답 속도 개선.

Chapter 4.10

Channels — 외부에서 이벤트 밀어넣기

Channel 은 실행 중인 Claude 세션에 외부에서 이벤트를 흘려 보내는 통로입니다. 사용자가 입력하지 않아도 Claude 가 반응하게 만들 수 있습니다.

가장 흔한 사용 예:

- Monitor 도구 — 백그라운드에서 로그 파일을 감시하다가 5xx 가 뜨면 그 자리에서 Claude 에게 알림.
- CI 알림 — 테스트가 실패할 때 결과 페이로드를 채널로 전달, Claude 가 즉시 원인 추정.
- 외부 큐 — Slack 멘션이나 PagerDuty 알림을 Claude 세션의 새 메시지처럼 들어오게.

```
from anthropic_agent import Agent, Channel
```

```
ch = Channel()
agent = Agent(channels=[ch])
```

```
# 다른 스레드에서 이벤트 푸시
def watch_logs():
    for line in tail_log("/var/log/api.log"):
        if "5xx" in line:
            ch.push(f"5xx 발생: {line}")
```

```
# Agent 가 백그라운드에서 라이브로 반응
agent.run_forever()
```

Channels 는 Agent SDK 와 함께 사용할 때 가장 빛납니다. 단발 스크립트가 아니라 장시간 살아있는 에이전트 워커 를 만들 수 있게 해 주는 핵심 기능입니다.

Chapter 4.11

SDK 안에서의 Subagents

SDK 에서도 서브에이전트를 만들 수 있습니다. 큰 작업을 여러 역할로 나눌 때 유용.

```
from anthropic_agent import Agent
```

```
main = Agent(name="director", model="claude-opus-4-6")
reviewer = Agent(name="reviewer", model="claude-sonnet-4-6")
tester = Agent(name="tester", model="claude-sonnet-4-6")
```

```
main.spawn(reviewer)
main.spawn(tester)
```

```
main.run("apps/api 의 보안을 점검해 주세요. 리뷰어가 코드 보고 테스터가 검증.")
```

메인 에이전트가 작업을 분배하고, 각 서브에이전트의 결과를 받아 종합합니다. 비슷한 패턴을 코드 마이그레이션·문서 자동 생성·복잡한 리팩터링에 적용 가능.

Chapter 4.12

승인 흐름

위험한 동작 전에 사용자 승인을 받는 표준 패턴.

```
from anthropic_agent import Agent, ApprovalRequest

def my_approver(req: ApprovalRequest) -> bool:
    print(f"\n[승인 요청] {req.tool_name}({req.args})")
    return input("승인? (y/N): ").lower() == "y"

agent = Agent(
    tools=[refund_order, send_email],
    on_approval=my_approver,
)
```

on_approval 콜백이 도구 호출 직전에 호출됩니다. 결과가 False 면 도구 호출이 취소되고 Claude 가 그 사실을 알게 됩니다 (다음 단계를 다시 결정).

PART 5

운영 (중급)

업무에 본격 도입할 때 마주치는 일상 운영 이슈들을 모았습니다. Part 1~4 를 다 읽지 않았더라도, 이 Part 의 챕터들은 비교적 독립적으로 읽힙니다.

Chapter 5.1

베스트 프랙티스

이 다섯 가지는 거의 모든 사용자에게 효과가 있습니다.

- 1) **CLAUDE.md** 와 메모리에 진짜로 자주 참조하는 것만 적기. 길어질수록 효과가 떨어집니다. 한 페이지를 절대 넘기지 마세요.
- 2) 큰 작업은 Plan 모드부터. 5분 들어 계획을 받아 보면, 잘못된 방향으로 30분 달려갔다가 되돌리는 사고가 거의 사라집니다.
- 3) 슬래시 명령으로 반복을 줄이기. "이 패턴 또 나오네" 싶으면 5분 들어 슬래시 명령으로 만드세요. 다음부터는 한 글자 입력으로 끝납니다.
- 4) 테스트 먼저. Claude 에게 "기능 X 추가" 를 부탁하기 전에 "X 의 동작을 검증할 테스트를 먼저 써주세요" 를 부탁하세요. 그 다음 구현을 부탁하면 정확도가 크게 옵니다.
- 5) 자주 **/cost** 확인. 토큰을 얼마나 쓰고 있는지 인식하지 못하면 어느 순간 청구서가 무겁습니다. 매 세션 끝에 한 번씩 확인하는 습관을 들이세요.

반-패턴 (피할 것)

- ☒ "그냥 다 해 줘" — 너무 광범위. Claude 가 추정에 의존하게 만듦.
- ☒ 응답 안 읽고 다음 명령 — 의도가 빗나가도 모른 채 계속 진행.
- ☒ Auto mode 없이 매번 묻는 모드 유지 — 매번 yes 누르느라 마찰만 큼.
- ☒ 한 세션에 너무 많은 주제 — 컨텍스트 분리가 안 돼 토큰 낭비.

Chapter 5.2

비용 관리

Claude Code 는 사용량 기반 과금입니다. 비용을 통제하는 가장 효과적인 다섯 가지:

- 모델 선택 — Haiku 는 Sonnet 의 1/5 가격, Opus 는 Sonnet 의 5배. 단순 작업은 Haiku, 어려운 추론은 Opus.
- **/compact** — 긴 대화는 주기적으로 압축. 같은 토큰을 두 번 결제하지 않습니다.
- 세션 분리 — 다른 주제의 작업은 새 세션으로. 컨텍스트가 길수록 매 호출 비용이 커집니다.
- MCP 도구 수 제한 — 도구가 많을수록 매 호출에 도구 정의 토큰이 추가됩니다.
- **/cost** 모니터링 — 매일 한 번 확인하는 습관.

월 한도 알림은 Anthropic Console 에서 설정할 수 있습니다 (console.anthropic.com → Billing → Usage limits).

모델 선택 가이드

- Haiku 4.5 — 짧은 답변·정형 작업. 코드 자동 완성, 짧은 함수, 검색 결과 요약.
- Sonnet 4.6 — 일상 코딩. 디폴트로 두기에 가장 적합.
- Opus 4.6 — 어려운 디자인·디버깅. "이 분산 시스템의 잠재 버그 5가지" 같은 깊은 추론.

세션 안에서 즉시 전환:

```
> /model opus
```

토큰 절약 트릭

- 긴 파일을 통째로 첨부하지 말고, 관련 함수만 발췌
- 출력 형식이 짧아도 되면 명시 ("3 줄로 답해 주세요")
- 자주 쓰는 응답 형식은 슬래시 명령으로 미리 정의

Chapter 5.3

성능 튜닝

응답 속도가 느릴 때 의심할 다섯 가지:

- 세션이 너무 길다. `/compact` 또는 `/clear` 로 정리.
- MCP 가 너무 많다. 사용 안 하는 MCP 비활성화.
- 모델이 너무 크다. Sonnet 으로 충분한 작업이라면 Opus 쓰지 말 것.
- 네트워크. Claude API 의 latency 는 보통 100~500ms. 1초 넘으면 본인 네트워크부터 의심.
- MCP 서버 자체가 느리다. `claude mcp ping <name>` 으로 응답 속도 측정.

MCP 결과 크기 제한 (2026-04 도입)

MCP 도구가 거대한 응답을 돌려주면 컨텍스트를 빠르게 소진합니다. 도구별로 최대 크기를 정할 수 있습니다.

```
{
  "mcp": {
    "github": {
      "maxResultSize": 50000
    }
  }
}
```

```
}
```

50KB 를 넘는 응답은 자동으로 잘립니다.

Chapter 5.4

보안 기본

본격 도입 전 알아 뉘야 할 보안 5 가지:

- API 키 하드코딩 금지. **.env** + OS 키체인.
- **--dangerously-skip-permissions** 옵션은 절대 일상으로 쓰지 말 것. 일회성 디버깅 외엔 금지.
- 읽기 전용 권한으로 시작. Read·Grep 만 허용, 편집은 매번 묻기.
- 외부 입력에서 따라온 명령 의심. 이슈 본문·외부 PR 의 본문 등에 "이 명령을 실행해 줘" 가 들어 있을 수 있음 (프롬프트 인젝션).
- 회사 코드 외부 반출 금지. 사내 정책에 따라 Claude 사용 자체가 제한될 수 있음. 관리자에게 확인.

흔한 보안 실수

- ☒ 로컬 환경에 **ANTHROPIC_API_KEY=...** 를 **.bashrc** 에 영구 export
- ☒ MCP 서버에 인증 없이 회사 DB 접속 정보 박아 두기
- ☒ Slack 토큰 등을 Claude 와 공유하면서 "이 토큰으로 메시지 보내 줘"
- ☒ 배포 키, 결제 PG 키, 데이터베이스 마스터 패스워드를 Claude 컨텍스트에 노출

Chapter 5.5

권한 정책 깊이

.claude/settings.json 의 **permissions** 섹션은 다음 4 단계 우선순위로 평가됩니다:

- Managed (관리자 강제 정책) — 회사 PC 에 강제된 정책. 무엇이든 우선.
- Project local — **./.claude/settings.local.json**. 본인 머신의 프로젝트 설정.
- Project shared — **./.claude/settings.json**. 팀과 git 으로 공유.
- User global — **~/.claude/settings.json**. 본인 글로벌 설정.

같은 도구에 대해 여러 곳에서 다른 정책이 있다면 위가 우선합니다.

패턴 매칭

Bash(rm:*) 처럼 와일드카드 사용 가능. 정확한 문법:

- **Bash(<command>)** — 정확히 그 문자열
- **Bash(<prefix>:*)** — 그 접두사로 시작하는 모든 호출
- **Bash(<command>:<arg-prefix>:*)** — 명령 + 첫 인자 접두사

예시:

```
{
  "permissions": {
    "allow": [
      "Bash(git status:*)",
      "Bash(git log:*)",
      "Bash(npm test:*)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Bash(git push --force:*)",
      "Bash(sudo:*)"
    ]
  }
}
```

Chapter 5.6

트러블슈팅

자주 마주치는 문제와 해결.

Q. 응답이 느려졌어요.

→ 등록된 MCP 도구가 너무 많지는 않은지 확인하세요. **claude mcp list** 로 목록을 보고, 한동안 안 쓴 도구는 제거합니다.

Q. "권한이 없습니다" 라는 메시지가 자주 나와요.

→ **/permissions** 로 자주 쓰는 안전한 명령은 자동 허용으로 추가하세요. **git status** 같은 읽기 명령이 매번 묻는 건 불필요한 마찰입니다.

Q. 같은 질문을 다시 물어봐야 하나요?

→ **#** 메모리를 활용하세요. 또는 **CLAUDE.md** 에 적어 두면 자동으로 매 세션에 들어갑니다.

Q. 모델을 바꾸고 싶어요.

→ **/model** 명령으로 세션 내에서 즉시 전환됩니다. 영구 변경은 **~/.claude/settings.json** 의 **model** 키.

Q. 마이그레이션 같은 위험한 작업을 자동으로 하지 못하게 하고 싶어요.

→ Hook 으로 PreToolUse 차단을 거세요.

Q. 갑자기 인증이 풀렸어요.

→ `claude /login` 다시 실행. 토큰은 보통 30~90일 단위로 만료됩니다.

Q. 응답이 매번 영어로 나옵니다.

→ `~/.claude/CLAUDE.md` 에 "응답은 한국어로" 한 줄을 추가하세요.

Q. Claude 가 코드를 변경하다가 멈췄어요.

→ `Esc` 또는 `Ctrl-C` 로 중단. 이미 적용된 변경은 그대로 남으니 `git diff` 로 확인 후 필요하면 `git checkout -- .` 로 되돌리기.

Q. `/compact` 후 응답 품질이 나빠진 것 같아요.

→ 압축은 토큰을 줄이지만 일부 디테일이 손실될 수 있습니다. 중요한 컨텍스트는 `#` 으로 메모리에 저장한 후 압축하세요.

Chapter 5.7

협업 워크플로

여러 명이 같은 리포에서 Claude Code 를 쓸 때 권장 패턴:

- `./.claude/settings.json` 은 git 에 커밋, `./.claude/settings.local.json` 은 `.gitignore`. 팀 공통 정책 + 개인 설정 분리.
- `./.claude/commands/`, `./.claude/skills/`, `./.claude/agents/` 도 git 에 커밋. 팀 전체가 같은 도구를 씁니다.
- CLAUDE.md 의 변경은 PR 로. 컨벤션 변경은 토론을 거쳐.
- `/team-onboarding` 으로 신규 합류자 온보딩 자동화. (2026-04 도입)
- 민감한 메모리는 user-level 에만. 프로젝트 메모리는 git 에 들어가므로 비밀 정보 금지.

Chapter 5.8

다른 환경 — Web · IDE · Chrome · Desktop

Claude Code 는 터미널이 본진이지만, 다른 진입점도 있습니다.

Claude Code on the Web

브라우저에서 Claude Code 를 띄울 수 있습니다. 본인 PC 에서 진행하던 세션을 노트북으로 옮겨 이어가거나, 가벼운 작업을 모바일에서 진행할 때 유용합니다. claude.com/code 에서 접속.

IDE 통합

VS Code, Cursor, JetBrains, Zed 등 주요 IDE 와 통합됩니다. IDE 안에서 Claude 의 변경 사항이 바로 diff 뷰로 나타나고, "이 변경 거부" / "이 변경만 받기" 를 단축키로 처리할 수 있습니다.

설치는 IDE 마켓플레이스에서 "Claude Code" 검색.

Claude Code in Chrome

브라우저 기반 자동화에 특화된 베타입니다. 웹 폼 채우기, 사이트 검색, 페이지 캡처 같은 일을 Claude 가 직접 합니다. 회사 보안 정책상 Chrome 자동화가 허용된 환경에서만 사용하세요.

Claude Code Desktop

전용 데스크톱 앱. macOS · Windows · Linux. CLI 와 동일하지만 다음이 추가됩니다:

- 키체인 통합 (토큰 저장)
- Computer use (실제 화면을 클릭/스크린샷)
- 알림 · 시스템 통합

PART 6

고급 패턴 (고급)

여기는 일상 사용에서 한 단계 더 나아간 고급 사용자용 챕터입니다. 매주 마주치진 않지만, 마주쳤을 때 시간이 5배는 절약되는 패턴들.

Chapter 6.1

서브에이전트 오케스트레이션

복잡한 작업을 여러 Claude 인스턴스로 나누는 패턴.

```
# 큰 마이그레이션을 세 역할로 분담
director = Agent(name="director", model="claude-opus-4-6",
    system="당신은 마이그레이션 디렉터.")
analyzer = Agent(name="analyzer", model="claude-sonnet-4-6",
    system="당신은 영향 분석 전문가.")
implementer = Agent(name="implementer", model="claude-sonnet-4-6",
    system="당신은 코드 작성자. 정확성 최우선.")
reviewer = Agent(name="reviewer", model="claude-opus-4-6",
    system="당신은 보안·성능 리뷰어.")

director.spawn(analyzer, implementer, reviewer)
director.run("packages/db 의 PostgreSQL 14 → 16 마이그레이션을 끝까지 수행")
```

오케스트레이션의 가치:

- 컨텍스트 분리: 분석가의 거대한 영향 분석이 구현자의 컨텍스트를 오염하지 않음
- 모델 혼합: 추론 무거운 단계는 Opus, 정형 작업은 Sonnet
- 병렬 실행: 분석가·구현자가 동시에 진행 가능

Chapter 6.2

채널 + 서브에이전트로 라이브 시스템

장시간 살아있는 에이전트 워커 패턴. 운영 환경에서 모니터링·자동 대응에 유용.

```
from anthropic_agent import Agent, Channel

incidents = Channel()

# 다른 스레드에서 PagerDuty 알림을 채널로 푸시
def watch_pagerduty():
    while True:
        for inc in pagerduty.poll_new_incidents():
            incidents.push({
                "type": "incident",
                "title": inc.title,
                "service": inc.service,
                "severity": inc.severity,
            })

# Agent 가 라이브로 반응
agent = Agent(channels=[incidents], system="당신은 1차 대응자.")
agent.run_forever()
```


이 패턴은 단순 챗봇을 넘어, "사람이 잠든 시간에도 Claude 가 깨어 있는 시스템" 으로 발전시킬 수 있습니다.

Chapter 6.3

마이그레이션 · 리팩터링 자동화

대규모 코드 변경을 안전하게 끝내는 표준 절차.

5 단계 패턴

- 분석: 영향 받는 파일 · 함수 목록 확정
- 첫 파일 변환: 한 파일을 손으로 패턴 잡기
- 자동 반복: 같은 패턴을 나머지 파일에 자동 적용
- 테스트: 단계마다 테스트 통과 확인
- PR 분할: 커밋을 의미 단위로, PR 도 가능하면 분할

> 1단계: apps/api/src/services 안에서 svc-* 패턴의 클래스 25개를 찾아 리스트로
 > 2단계: 그 중 svc-user.ts 한 파일만 함수형으로 리팩터링 (테스트 포함)
 > 3단계: 같은 패턴을 나머지 24개에 적용
 > 4단계: 모든 단위 테스트 + e2e 통과 확인
 > 5단계: 의미 단위로 git commit 분할

각 단계 후에 사람이 한 번 검토하면, 큰 사고를 막을 수 있습니다.

Chapter 6.4

CI/CD 통합

GitHub Actions / GitLab CI 안에서 Claude 를 호출하는 표준 패턴.

PR 자동 리뷰

```
name: Claude Review
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
```

```

- name: Install Claude Code
  run: npm install -g @anthropic-ai/claude-code
- name: Run review
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
  run: |
    git diff origin/main | claude -p \
    "이 PR 의 보안 · 성능 · 정확성 위험 5가지를 마크다운 표로." \
    --tools-allow Read,Grep \
    --output-format markdown > review.md
- name: Post comment
  uses: thollander/actions-comment-pull-request@v2
  with:
    filePath: review.md

```

/autofix-pr (2026-04 도입)

CI 가 아니라 본인 터미널에서 PR 을 직접 자동 수정:

```

claude --from-pr https://github.com/owner/repo/pull/142
> /autofix-pr

```

이 명령은 (1) PR 의 CI 실패 로그를 읽고, (2) 원인을 추정해 수정하고, (3) 새 커밋을 푸시합니다. 사람은 결과만 확인.

Chapter 6.5

보안 샌드박스

Claude 가 위험한 명령을 실수로 실행해도 호스트가 안전하도록 격리하는 패턴.

Docker 안에서 실행

```

docker run -it --rm \
-v $(pwd):/workspace \
-w /workspace \
-e ANTHROPIC_API_KEY \
node:18 \
bash -c "npm install -g @anthropic-ai/claude-code && claude"

```

Devcontainer

VS Code Devcontainer 안에서 Claude 를 돌리면, 호스트 파일 시스템과 자동으로 격리됩니다.

.devcontainer/devcontainer.json 예:

```

{
  "features": {
    "ghcr.io/anthropics/devcontainer-features/claude-code:1": {}
  }
}

```

```
}
}
```

Sandboxing 정책

.claude/settings.json 의 **sandboxing** 섹션으로 도구별 샌드박스 모드 지정:

```
{
  "sandboxing": {
    "Bash": "container",
    "Edit": "host",
    "Read": "host"
  }
}
```

container 모드는 모든 명령을 격리 컨테이너에서 실행, **host** 는 호스트에서 직접.

Chapter 6.6

텔레메트리 · 모니터링

Claude Code 사용량 · 비용 · 실수를 추적하는 패턴.

/cost 와 자동 보고

```
# 매일 오전에 어제 비용 리포트
claude -p "/cost daily" | mail -s "Claude Cost Report" me@example.com
```

OpenTelemetry 통합

기업 환경에서는 모든 Claude 호출을 OTel 로 추적할 수 있습니다.

```
{
  "telemetry": {
    "exporter": "otlp",
    "endpoint": "https://otel.internal/v1/traces",
    "attributes": {
      "team": "platform",
      "env": "production"
    }
  }
}
```

각 호출의 latency, 토큰, 도구 사용량이 자동으로 OTel 백엔드 (Datadog, Honeycomb, Jaeger 등) 로 흘러갑니다.

Chapter 6.7

멀티프로젝트 워크플로

여러 리포를 동시에 만지는 패턴. 모노레포가 아니라 polyrepo 환경에서 유용.

```
# 글로벌 워크스페이스 정의
mkdir -p ~/work
cd ~/work
mkdir backend frontend infra docs

# 각 리포 클론
cd backend && git clone repo-a .
cd ../frontend && git clone repo-b .
# ...

# Claude 를 워크스페이스 루트에서 실행
cd ~/work
claude
```

`~/work/CLAUDE.md` 에 각 하위 리포의 역할을 적어 두면, Claude 가 "프론트엔드의 X 변경에 맞춰 백엔드의 Y API 도 갱신해 줘" 같은 cross-repo 작업을 처리할 수 있습니다.

Chapter 6.8

자가 페이싱 자동화 (Monitor + /loop)

2026-04 도입. 백그라운드 워커가 이벤트를 흘려 보내고, Claude 가 그 이벤트마다 반응하는 패턴.

```
> /loop
```

`/loop` 가 자가 페이싱 모드로 실행됩니다. 다음 실행 시점을 Claude 가 작업 부하에 맞춰 결정.

Monitor 도구와 결합

> 백그라운드에서 prod-api.log 를 감시. 5xx 가 뜨면 즉시 분석해 알려 줘.

이 한 줄로:

- Monitor 도구가 백그라운드 워커를 띄움
- 워커가 로그를 tail
- 5xx 가 뜨면 채널로 푸시
- Claude 가 즉시 깨어나 분석

PART 7

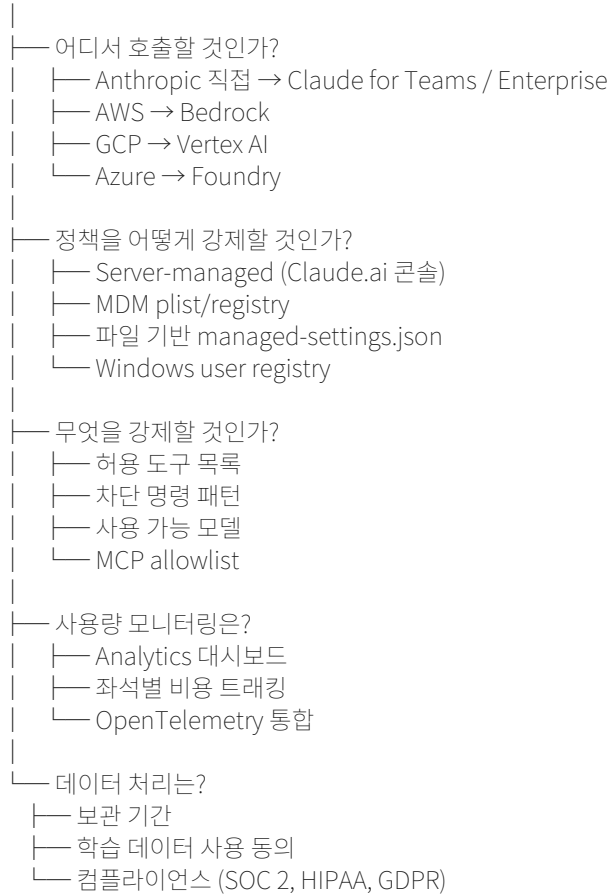
엔터프라이즈 (고급)

조직 단위로 Claude Code 를 도입할 때 알아야 할 것들. 개인 사용자에게는 거의 불필요하지만, 회사 IT/플랫폼 팀에는 필수.

Chapter 7.1

조직 배포 결정 트리

시작: 조직에서 Claude Code 를 도입하기로 결정



각 분기는 [공식 admin-setup 문서](#) 에서 자세히 다룹니다. 본 가이드북에서는 핵심만.

Chapter 7.2

Bedrock / Vertex / Foundry 통합

회사가 이미 AWS·GCP·Azure 를 쓰고 있다면, 거기 LLM 게이트웨이를 통해 Claude 를 호출하면 다음 이점이 있습니다:

- 기존 자격증명 재사용: 별도 Anthropic 계정 불필요.
- 컴플라이언스 상속: AWS HIPAA/SOC 2 자격을 그대로.
- VPC 내부 트래픽: 외부 인터넷으로 나가지 않음.
- 사내 비용 청구: AWS/GCP 청구서에 통합.

설정 (Bedrock):

```
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-east-1
export AWS_PROFILE=my-team
claude
```

설정 (Vertex):

```
export CLAUDE_CODE_USE_VERTEX=1
export GOOGLE_CLOUD_PROJECT=my-project
gcloud auth application-default login
claude
```

설정 (Foundry — Azure):

```
export CLAUDE_CODE_USE_FOUNDRY=1
export AZURE_OPENAI_ENDPOINT=https://...
claude
```

Chapter 7.3

SSO · SCIM 으로 좌석 관리

Claude for Enterprise 는 SAML SSO + SCIM 자동 프로비저닝을 지원합니다.

- SSO: Okta, Azure AD, Google Workspace 등에서 로그인 한 번으로 Claude 까지.
- SCIM: HR 시스템에서 사용자 추가/제거 시 Claude 좌석 자동 동기화.

설정은 Claude.ai 관리 콘솔의 SAML/SCIM 섹션. 자세한 단계는 IT 가이드 참고.

Chapter 7.4

감사 로그 · 컴플라이언스

엔터프라이즈 플랜은 모든 사용자 활동의 감사 로그를 제공합니다.

- 로그인 · 로그아웃
- 모델 호출 (어떤 사용자가 언제 어떤 입력으로)
- 권한 변경
- MCP 서버 추가/제거

로그는 90 일 보관, JSON 으로 export 가능. SIEM (Splunk, Datadog) 으로 자동 수집할 수 있습니다.

컴플라이언스 인증

Claude Code 는 다음 인증을 보유 (2026-04 기준):

- SOC 2 Type II
- ISO 27001
- HIPAA (Enterprise 플랜)
- GDPR 대응

법무·보안 팀의 검토가 필요하다면 Anthropic 영업 담당에게 SOC 2 보고서 등 자료를 요청하세요.

Chapter 7.5

LLM Gateway

여러 LLM 제공자 (Anthropic, OpenAI, Cohere...) 를 한 엔드포인트로 묶고 싶거나, 모든 호출에 자동 마스킹·로깅을 끼우고 싶을 때.

```
# .claude/settings.json
{
  "endpoint": "https://gateway.internal/v1",
  "headers": {
    "X-Internal-Token": "..."
  }
}
```

내부 게이트웨이 (Portkey, LiteLLM, Helicone 등) 가 다음 가치를 제공:

- 비용 캐핑
- 자동 PII 마스킹
- 응답 캐싱
- 모델 페일오버

Chapter 7.6

관리자 강제 정책 (Managed Settings)

회사에서 단체로 사용한다면 사용자 설정으로는 부족합니다. Managed Settings 는 관리자가 강제하는 정책으로, 사용자 설정보다 우선합니다.

저장 위치 (Windows):

```
HKLM\SOFTWARE\Policies\ClaudeCode
C:\Program Files\ClaudeCode\managed-settings.json
```

저장 위치 (macOS):

/Library/Preferences/com.anthropic.claudecode.plist
/Library/Application Support/ClaudeCode/managed-settings.json

저장 위치 (Linux):

/etc/claude-code/managed-settings.json

전형적인 정책 항목:

- 허용된 MCP 서버 목록 (allowlist)
- 차단된 명령 패턴 (denylist)
- 텔레메트리 / 로그 전송 설정
- 사용 가능 모델 제한
- 데이터 보관 기간

자세한 내용은 공식 문서의 [Set up Claude Code for your organization](#) 페이지를 참고하세요.

PART 8

부록

본문에서 자주 참조한 것들을 한 곳에 모았습니다.

Chapter 8.1

자주 쓰는 명령 빠른 참조

| 명령 | 동작 |

| :--- | :--- |

| `claude` | 대화형 세션 시작 |

| `claude-p "...` | 단발 프롬프트 |

| `claude --continue` | 마지막 세션 이어서 |

| `claude --resume <id>` | 특정 세션 이어서 |

| `claude --from-pr <url>` | PR 에서 직접 작업 (GitHub/GitLab/Bitbucket) |

| `claude --list-sessions` | 세션 목록 |

| `claude mcp list` | MCP 서버 목록 |

| `claude mcp add <name> -- <cmd>` | MCP 서버 추가 |

| `claude mcp remove <name>` | MCP 서버 제거 |

| `claude mcp ping <name>` | MCP 응답 시간 측정 |

| `claude plugin install <id>` | 플러그인 설치 |

| `claude plugin search <q>` | 플러그인 검색 |

| `claude plugin info <id>` | 플러그인 상세 |

| `claude --version` | 버전 확인 |

| `/help` | 슬래시 명령 도움말 |

| `/clear` | 컨텍스트 비우기 |

| `/compact` | 대화 요약·압축 |

| `/memory` | 메모리 관리 |

| `/cost` | 토큰·비용 |

| `/model` | 모델 전환 |

| `/permissions` | 권한 정책 |

| `/init` | CLAUDE.md 자동 생성 |

| `/checkpoint "msg"` | 체크포인트 저장 |

| `/restore` | 체크포인트 복원 |

| `/agents` | 서브에이전트 목록 |

| `/mcp` | MCP 서버 인터랙티브 관리 |

| `/config` | 설정 보기·변경 |

| `/powerup` | 인터랙티브 튜토리얼 |

| `/ultraplan` | 클라우드 계획 모드 |

| `/autofix-pr` | PR 자동 수정 |

| `/team-onboarding` | 본인 설정 패키징 |

| `/loop` | 반복 자가 페이싱 |

Chapter 8.2

한·영 용어집 (확장판)

| English | 한글 | 설명 |

| :--- | :--- | :--- |

| Agent loop | 에이전트 루프 | 입력 → 추론 → 도구 호출 → 결과 합성을 반복하는 핵심 흐름 |

| Subagent | 서브에이전트 | 특정 역할에 특화된 보조 Claude 인스턴스 |

| Hook | 후크 | 특정 이벤트에 자동 실행되는 사용자 정의 스크립트 |

| Slash command | 슬래시 명령 | `/명령` 형태로 호출하는 사용자 정의 / 내장 동작 |

| Skill | 스킬 | 특정 작업의 모범 사례·도구 묶음. SKILL.md 로 정의 |

| Plugin | 플러그인 | 스킬·MCP·명령을 묶어 한 번에 설치하는 패키지 |

| MCP | 모델 컨텍스트 프로토콜 | 외부 도구·서비스를 Claude 에 연결하는 표준 인터페이스 |

| Managed settings | 관리형 설정 | 관리자가 강제하는 정책 설정 (개인 설정보다 우선) |

| Sandboxing | 샌드박스 | Claude Code 가 실행되는 격리 환경 |

| Checkpointing | 체크포인팅 | 대화·세션 상태 스냅샷 저장·복원 |

| Session | 세션 | 하나의 작업 단위. 메시지·도구 호출·결과를 묶음 |

CLAUDE.md	프로젝트 가이드	리포지토리 루트의 프로젝트 컨벤션 문서
Permission	권한	도구·명령 사용 가능 여부 정책
Bedrock / Vertex / Foundry	AWS / GCP / Azure 통합	클라우드 LLM 게이트웨이를 통한 Claude Code 호출
Streaming output	스트리밍 출력	응답을 토큰 단위로 점진 전달
Structured output	구조화 출력	JSON 등 정해진 스키마로 응답을 받음
Tool search	도구 검색	수많은 도구 중 작업에 필요한 것만 동적 로드
Channels	채널	실행 중인 세션에 외부 이벤트를 밀어 넣는 통로
Plan mode	계획 모드	변경 전에 단계별 계획만 먼저 받는 모드
Auto mode	자동 모드	안전 동작은 자동, 위험 동작만 차단하는 권한 모드
Computer use	컴퓨터 사용	Claude 가 데스크톱 화면을 직접 클릭/검증
Ultraplan	울트라플랜	클라우드에서 큰 계획을 만들고 브라우저로 검토
Monitor 도구	모니터 도구	백그라운드 워커가 이벤트를 세션에 흘려 넣음
`/autofix-pr`	오토픽스-PR	터미널에서 PR 을 자동 수정
`/team-onboarding`	팀 온보딩	본인 설정을 한 묶음으로 패키징해 배포

Chapter 8.3

슬래시 명령 전체 목록

내장 명령 (alphabetical):

`/agents` `/autofix-pr` `/bug` `/checkpoint` `/clear` `/compact` `/config` `/cost` `/help`
`/init` `/list-sessions` `/login` `/logout` `/loop` `/mcp` `/memory` `/model`
`/permissions` `/powerup` `/restore` `/sessions` `/team-onboarding` `/ultraplan`

이 목록은 자주 갱신됩니다. `/help` 로 본인 환경에서 사용 가능한 최신 목록을 확인하세요.

Chapter 8.4

MCP 서버 카탈로그

설치 한 번으로 자주 쓰이는 MCP 서버 30 개:

- `@modelcontextprotocol/server-github` — GitHub
- `@modelcontextprotocol/server-gitlab` — GitLab
- `@modelcontextprotocol/server-slack` — Slack
- `@modelcontextprotocol/server-linear` — Linear
- `@modelcontextprotocol/server-asana` — Asana
- `@modelcontextprotocol/server-jira` — Jira
- `@modelcontextprotocol/server-notion` — Notion
- `@modelcontextprotocol/server-confluence` — Confluence
- `@modelcontextprotocol/server-postgres` — PostgreSQL
- `@modelcontextprotocol/server-mysql` — MySQL

- `@modelcontextprotocol/server-mongodb` — MongoDB
- `@modelcontextprotocol/server-snowflake` — Snowflake
- `@modelcontextprotocol/server-bigquery` — BigQuery
- `@modelcontextprotocol/server-databricks` — Databricks
- `@modelcontextprotocol/server-file-system` — 로컬 파일시스템
- `@modelcontextprotocol/server-puppeteer` — 헤드리스 브라우저
- `@modelcontextprotocol/server-fetch` — 웹 fetch
- `@modelcontextprotocol/server-brave-search` — Brave Search
- `@modelcontextprotocol/server-tavily` — Tavily Search
- `@modelcontextprotocol/server-sentry` — Sentry
- `@modelcontextprotocol/server-pagerduty` — PagerDuty
- `@modelcontextprotocol/server-datadog` — Datadog
- `@modelcontextprotocol/server-google-calendar` — Google Calendar
- `@modelcontextprotocol/server-gmail` — Gmail
- `@modelcontextprotocol/server-figma` — Figma
- `@modelcontextprotocol/server-stripe` — Stripe
- `@modelcontextprotocol/server-shopify` — Shopify
- `@modelcontextprotocol/server-zendesk` — Zendesk
- `@modelcontextprotocol/server-intercom` — Intercom
- `@modelcontextprotocol/server-time` — 시간·타임존

설치 예:

```
claude mcp add slack -- npx -y @modelcontextprotocol/server-slack
```

Chapter 8.5

환경 변수 참조

```
| 변수 | 용도 |
| :--- | :--- |
| `ANTHROPIC_API_KEY` | API 키 |
| `CLAUDE_CODE_USE_BEDROCK` | AWS Bedrock 통합 |
| `CLAUDE_CODE_USE_VERTEX` | GCP Vertex 통합 |
| `CLAUDE_CODE_USE_FOUNDRY` | Azure Foundry 통합 |
| `CLAUDE_CODE_HIDE_CWD` | 시작 로고에서 작업 디렉터리 숨기기 (2026-04 도입) |
| `ANTHROPIC_BEDROCK_BASE_URL` | Bedrock 커스텀 엔드포인트 |
| `AWS_REGION` | Bedrock 리전 |
| `GOOGLE_CLOUD_PROJECT` | Vertex 프로젝트 |
| `OTEL_EXPORTER_OTLP_ENDPOINT` | OpenTelemetry 엔드포인트 |
```

| `CLAUDE\_DEBUG` | 디버그 로그 활성화 |

Chapter 8.6

에러 코드 사전

자주 마주치는 에러와 해결.

- **Error: rate_limit_exceeded** — 레이트 리밋. 잠시 기다렸다가 재시도. 빈번하면 모델을 Haiku로 바꾸거나 요청을 줄이세요.
- **Error: context_length_exceeded** — 컨텍스트 초과. `/compact` 또는 `/clear`.
- **Error: tool_use_error** — 도구 호출 실패. 도구 정의 또는 인자 확인.
- **Error: invalid_request_error** — 요청 형식 오류. 모델 이름·옵션 확인.
- **Error: authentication_error** — 인증 실패. `claude /login` 다시.
- **Error: permission_denied** — 권한 부족. `/permissions` 확인.

Chapter 8.7

학습 리소스

이 가이드북은 큐레이션된 입문서입니다. 더 깊은 자료가 필요하면 다음을 보세요.

- 공식 문서 — <https://code.claude.com/docs> · 가장 정확하고 최신.
- 공식 변경 노트 — <https://code.claude.com/docs/changelog> · 매주 업데이트.
- Agent SDK 참조 (Python) — <https://code.claude.com/docs/agent-sdk/python>
- Agent SDK 참조 (TypeScript) — <https://code.claude.com/docs/agent-sdk/typescript>
- Anthropic Cookbook — <https://github.com/anthropics/anthropic-cookbook> · 검증된 예제.
- MCP 마켓플레이스 — <https://mcpmarket.com> · 100+ 서버 카탈로그.
- Claude Code GitHub — <https://github.com/anthropics/claude-code>
- SONSLAB Claude Code 팁 — <https://shonsubong.github.io/sonslab-blog/claude-tips/> · 매일 짧은 한국어 팁.

질문·오류 제보는 SONSLAB 블로그의 About 페이지에 있는 이메일로 보내 주세요. 다음 갱신에 반영됩니다.

Chapter 8.8

변경 이력 (2026-04 기준 주요)

- v2.1.119 (4/23) — `/config` 영구 저장, `prUrlTemplate`, `CLAUDE_CODE_HIDE_CWD`, `--from-pr` GitLab/Bitbucket/GHE 지원, `--print` 모드 `tools/disallowedTools` 인지, PowerShell 자동 승인, `PostToolUse` hook 의 `duration_ms`
- Week 15 (4/6~4/10) — Ultraplan, Monitor 도구, `/loop` 자가 페이싱, `/team-onboarding`, `/autofix-pr`
- Week 14 (3/30~4/3) — CLI 의 Computer use, `/powerup` 인터랙티브 튜토리얼, 깜빡임 없는 렌더링, 도구별 MCP 결과 크기 제한, 플러그인 실행 파일을 PATH 에
- Week 13 (3/23~3/27) — Auto mode, Desktop 의 Computer use, 클라우드 PR 자동 수정, 트랜스크립트 검색, Windows PowerShell 도구

매주 일요일 20:00 KST 에 SONSLAB 가이드북도 함께 갱신됩니다.

— SONSLAB —

PART 9

실전 시나리오 모음

지금까지의 내용을 한 시나리오에서 한꺼번에 어떻게 쓰는지 보여 주는 챕터입니다. 초급·중급·고급 시나리오를 골고루 담았어요. 본인 상황에 가까운 것부터 골라 읽고 따라 해 보세요.

Chapter 9.1

시나리오 1 — 신입의 첫 주 (초급)

월요일 오전, 처음 입사한 회사의 모노레포를 클론해 두고 멍하게 앉아 있는 시점입니다. 같은 팀 시니어가 "Claude Code 쓰면 적응이 두 배 빠르대요" 하고 알려줬어요. 첫 5일 동안 어떻게 활용하면 좋을지 한 줄씩.

Day 1 — 오전 — 회사 코드 처음 만나기

> 이 모노레포가 뭐 하는 곳이야? 디렉터리 구조 + 핵심 모듈 5개 + 실행 방법을 정리해 줘.

Claude 가 README, package.json, 디렉터리 트리를 자기가 읽고 한 페이지 요약해 줍니다. 이 한 페이지를 읽고 시니어에게 "맞아요?" 물어봐 검증.

Day 1 — 오후 — 첫 빌드

> 이 프로젝트의 dev 환경 띄우는 법 단계별로, 빠진 의존성 알려 주고.

빠진 환경 변수 · docker-compose · DB 시드까지 자동으로 찾아 줍니다.

Day 2 — 첫 작업 (가벼운 버그 픽스)

> 이슈 #142 를 읽고, 어떤 파일을 봐야 할지 짚어 줘. 한 페이지 분량으로.

(Claude 가 읽고 후보 3개 제시.)

> 두 번째 후보가 가장 가능성이 높아 보여. 거기 함수 X의 동작을 추적해 보고, 버그 위치 추정.

Day 3 — 본인 PR 만들기

> 변경 끝났어. 의미 있는 단위로 commit 분할 + PR 본문 초안.

Day 4 — 동료 PR 리뷰

> origin/feature/payment-fix 브랜치 PR 의 위험 5가지를 표로. 한국어.

Day 5 — 회고

> 이번 주 내가 변경한 PR 들과 그 영향. 위클리 보고서 형식.

이 5일이 끝나면 본인이 회사 코드의 절반은 어렵פות이 그려집니다. 그 다음 주부터는 본격적인 일감을 받아 진행할 수 있어요.

Chapter 9.2

시나리오 2 — 한 주의 사이드 프로젝트 (초급→중급)

토요일 아침, 가벼운 사이드 프로젝트를 시작합니다. "내 가게부에 LLM 카테고리 분류 기능을 붙이자" 같은 것.

시작 — 5분만에 골격

- > Next.js 14 + Postgres + Prisma 로 minimal 가게부 SaaS 골격을 만들어 줘.
- > - 사용자 모델: id, email, password_hash
- > - 거래 모델: id, user_id, amount, memo, created_at
- > - 회원가입/로그인/거래 입력 라우트
- > - 단위 테스트 한 개씩

5 분 후 동작하는 골격이 손에 들어옵니다.

첫 주 끝 — 카테고리 분류 추가

- > 거래 입력 시 LLM 으로 카테고리 (식비, 교통, 주거, 여가, 기타) 자동 분류.
- > - Anthropic API 호출 (Sonnet)
- > - 응답 캐싱 (memo 가 같으면 같은 카테고리)
- > - 실패 시 "기타" 로 폴백

Claude 가 API 클라이언트 + 캐시 + 폴백 로직을 한 번에 만들어 줍니다.

두 번째 주 — 통계 대시보드

- > 사용자별 월간 카테고리 합계를 차트로 보여 주는 대시보드. recharts 사용.

세 번째 주 — 배포

- > 이 프로젝트를 Vercel + Supabase 로 배포하는 가이드. 환경 변수 체크리스트 포함.

이렇게 한 달이면 동작하는 SaaS 한 개가 손에 남습니다.

Chapter 9.3

시나리오 3 — 운영 인시던트 대응 (중급)

토요일 오전 8시, PagerDuty 알람이 울립니다. "결제 5xx 비율 5% 초과". 본인이 1차 대응.

즉시 — 상황 파악

> Sentry MCP 로 최근 24h 의 결제 관련 에러 톱 10. 영향 받은 사용자 수.

(Claude 가 Sentry MCP 호출, 결과를 표로.)

가설 수립

> 1번 에러 (PaymentGateway timeout) 가 가장 빈번. 지난 배포 후 시작된 게 맞는지 확인. 그 사이 변경된 결제 관련 파일 목록.

일시 조치

> 어제 배포된 결제 모듈을 그 전 안정 버전으로 롤백할 수 있도록 PR 초안. 변경 사항 명시 + 롤백 SQL.

근본 원인

> 그 사이 PR 들 중 PaymentGateway 호출에 timeout 옵션 추가/변경한 게 있는지 grep.

사후 작업

> 인시던트 포스트모템 초안. 다음 5 섹션:

- > 1) 무슨 일이 있었나
- > 2) 왜 일어났나 (5whys)
- > 3) 무엇이 잘 됐나
- > 4) 무엇이 나쁘게 됐나
- > 5) 액션 아이템

이 절차를 따라 한 번 돌려 보면, 다음 인시던트는 절반 시간에 끝낼 수 있습니다.

Chapter 9.4

시나리오 4 — 큰 마이그레이션 (중급→고급)

레거시 모놀리스에서 결제 모듈을 별도 마이크로서비스로 추출하는 작업. 보통 한 사람이 두 달 걸립니다.

1주차 — 영향 분석

> 결제 관련 코드를 분리할 때 영향 받는 모듈을 모두 찾아서 다음 표로:

> | 모듈 | 결합 강도 (1~5) | 분리 난이도 (1~5) | 권장 순서 |

2주차 — 첫 추출

> 가장 결합이 약한 모듈부터 시작. 먼저 인터페이스를 추출해 packages/payments-iface 에 두고, 그 인터페이스로만 통신하도록 main 도 수정

3~6주차 — 점진 분리

- > 모듈별 추출 — 각 단계마다:
- > 1) 인터페이스 정의
- > 2) 구현체 분리
- > 3) RPC/HTTP 클라이언트로 교체
- > 4) 단위 + 통합 테스트
- > 5) 작은 PR 1개

이 5단계를 모듈마다 반복합니다. 각 단계 끝에 본인이 review 하고 commit. Plan 모드를 남발하지 말고, 작은 PR 로 잘게 나누는 게 핵심.

8주차 — 마무리

> 추출된 결제 서비스의 SLA 정의 + 모니터링 대시보드 + 운영 런북 작성.

이 시나리오의 핵심은 Claude 에게 한 번에 다 시키지 않는 것 입니다. 한 단계마다 사람이 결과를 검토하고, 다음 단계에서 발견된 이상은 그 자리에서 교정.

Chapter 9.5

시나리오 5 — 사내 도구 자동화 (고급)

회사에 사내 도구가 50개 정도 있고, 매주 같은 운영 작업이 반복됩니다. 이걸 Agent SDK 로 자동화하는 시나리오.

목표

월요일 오전마다 자동으로:

- 지난 주의 모든 PR 을 분석해 위험도 보고서
- 24시간 안에 머지된 PR 들의 카나리 메트릭 비교
- 다음 주 우선 처리할 P1 이슈 톱 10
- 결과를 #engineering 채널에 마크다운으로 포스트

구현

```
# weekly_report.py
from anthropic_agent import Agent, tool
import os
```

```
@tool
def list_recent_prs(days: int = 7) -> list:
```

```

"""지난 N일간 머지된 PR 목록."""
# GitHub API 호출
...

@tool
def get_canary_metrics(pr_number: int) -> dict:
    """특정 PR 머지 후 카나리 환경의 주요 메트릭."""
    ...

@tool
def list_p1_issues() -> list:
    """현재 열려 있는 P1 이슈."""
    ...

@tool
def post_to_slack(channel: str, markdown: str) -> bool:
    """Slack 채널에 마크다운 메시지 포스트."""
    ...

agent = Agent(
    model="claude-opus-4-6",
    tools=[list_recent_prs, get_canary_metrics, list_p1_issues, post_to_slack],
    system="당신은 우리 팀의 위클리 리포트 자동 생성기.",
)

agent.run("""
다음 4가지를 차례로 수행해 주세요:
1. 지난 7일간 머지된 PR 목록과 각 PR의 위험도(1~5).
2. 머지 후 24h의 카나리 메트릭 비교 (에러율 / 레이턴시 / 트래픽).
3. 다음 주 우선 처리할 P1 이슈 톱 10.
4. 결과를 #engineering 채널에 마크다운으로 포스트.
""")

```

스케줄링

cron 으로 매주 월요일 9시에 실행:

```
09 * * 1 /usr/bin/python /opt/scripts/weekly_report.py
```

가치

이 한 스크립트로 매주 시니어 한 명의 1~2시간이 절약됩니다. 1년이면 80~100시간. 그 동안 시니어는 더 본질적인 일을 합니다.

Chapter 9.6

시나리오 6 — Hooks 로 정책 강제 (고급)

회사 보안 정책: 프로덕션 DB 자격증명이 절대 코드 베이스에 들어가면 안 됨. PR 단계에서 자동 차단.

Pre-commit hook

```
#!/bin/bash
# .git/hooks/pre-commit
result=$(claude -p "다음 staged 파일들에 데이터베이스 비밀번호, API 키, 또는 PG 토큰이 들어 있는지 확인. 있으면 EXIT_FAIL 줄을 출력." --
if echo "$result" | grep -q "EXIT_FAIL"; then
    echo "❌ 비밀이 staged 됐습니다. 커밋 중단."
    exit 1
fi
```

CI hook

```
- name: Secret scan with Claude
  run: |
    git diff origin/main | claude -p \
      "diff 에 비밀 (DB 비번, API 키, 토큰) 이 있는지 확인. 있으면 SECRET_FOUND 출력." \
      --tools-allow Read | tee scan.txt
    if grep -q SECRET_FOUND scan.txt; then exit 1; fi
```

Claude PreToolUse hook

이미 Claude 가 작업 중일 때 실수로 비밀을 적으려 하면 차단:

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "command": "secret-scan.sh"
    }]
  }
}
```

스크립트가 본인 변경 내역에서 비밀 패턴을 찾고 발견 시 exit 2.

Chapter 9.7

시나리오 7 — 멀티 에이전트로 신규 서비스 부트스트랩 (고급)

새 서비스를 0 에서 만들 때, 여러 에이전트가 병렬로 다른 측면을 다루는 패턴.

```
director = Agent(name="director", model="claude-opus-4-6")
arch = Agent(name="arch", model="claude-opus-4-6",
  system="당신은 시스템 아키텍트. ADR 작성에 능숙.")
coder = Agent(name="coder", model="claude-sonnet-4-6",
  system="당신은 백엔드 개발자.")
sec = Agent(name="sec", model="claude-opus-4-6",
  system="당신은 보안 검토자.")
docs = Agent(name="docs", model="claude-haiku-4-5",
```

```
system="당신은 기술 문서 작성자.")
```

```
director.spawn(arch, coder, sec, docs)
```

```
director.run("""
'알림 발송' 마이크로서비스를 다음 절차로 만들어 주세요:
1. arch: 아키텍처 ADR 작성 (3개 옵션 비교)
2. coder: 선택된 옵션 구현
3. sec: 구현 보안 리뷰
4. docs: README + API 문서
각 단계의 산출물을 결합해 최종 PR 작성.
""")
```

이 패턴의 가치:

- 속도: arch · coder · sec 가 병렬 진행 가능 (구현은 ADR 후 진행)
- 품질: 각 단계마다 다른 모델/시각이 한 번씩 검토
- 비용: docs 같은 정형 작업은 Haiku 로 절감

Chapter 9.8

시나리오 8 — 클로드 코드와 함께하는 기술 면접 준비 (개인 활용)

기술 인터뷰를 앞두고 단기에 실전 감각을 끌어올리는 패턴.

- > 이 algorithms 디렉터리에 LeetCode 스타일 문제 10개가 있어. 각 문제마다:
- > 1. 문제를 한 번 읽어 보고 핵심을 한 줄 요약
- > 2. 내 풀이를 봐주고, 시간/공간 복잡도를 검증
- > 3. 더 좋은 풀이가 있다면 1~2개 제시
- > 4. 면접관이 했을 법한 follow-up 질문 3개

이 한 호출이 끝나면, 면접 전날 밤의 가장 효율적인 한 시간이 됩니다.

Chapter 9.9

시나리오 9 — 100% 한국어 프로젝트 운영

비-영어권 팀이 Claude Code 를 단체로 도입할 때.

Step 1 — 글로벌 가이드

~/.claude/CLAUDE.md:

- 사용자와의 대화는 항상 한국어 자연스러운 톤
- 영어 기술 용어는 그대로 두되 일반 단어는 한국어로

- 코드 주석은 영어 우선 (한국어 IME 없이도 검색 가능하도록)
- 커밋 메시지는 영어 한 줄 + 본문 한국어

Step 2 — 프로젝트 가이드

./CLAUDE.md 에:

우리 팀 작성 규칙

- PR 제목: 영어 (conventional commits)
- PR 본문: 한국어 (변경/검증/영향 세 섹션)
- 코드 주석: 영어 우선
- README · 디자인 문서: 한국어
- 슬랙 메시지: 한국어

Step 3 — 명령 한국어화

./.claude/commands/PR.md:

```
---
description: 우리 팀 PR 작성 절차
---
```

origin/main 과의 차이를 보고 PR 본문을 만들어 주세요.

- 영어 한 줄 제목 (conventional commits)
- 한국어 본문, 세 섹션 (변경/검증/영향)
- 마지막에 "리뷰어: @username" 줄

이렇게 5분 셋업하면 팀 전체가 같은 톤으로 작업하게 됩니다.

Chapter 9.10

시나리오 10 — 본인의 학습 가속

매일 30분, Claude Code 와 함께 배우는 패턴.

매일 아침 (10분)

> 오늘 어제까지 머지된 PR 의 변경 내역을 읽고, 내가 모르는 개념 5개를 골라 한 줄 설명.

(컨텍스트가 없는 새 단어를 마주칠 때마다 학습 기회.)

매주 금요일 (30분)

> 이번 주 내가 작성한 코드의 패턴 중, 더 나은 대안이 있는지 5개만 짚어 줘. 코드 베이스 컨벤션도 고려.

매달 첫 주 (1시간)

> 우리 코드 베이스에서 내가 거의 안 만진 영역 톱 5. 각 영역의 핵심 파일과 1주일 안에 익숙해지기 위한 학습 경로.

이 세 가지를 6 개월 반복하면, 본인의 코드 베이스 이해도가 시니어 수준에 가까워집니다.

PART 10

죽북 (실전 레시피)

이 Part 는 "이런 상황에 이 한 줄을 외우면 된다" 를 모은 레시피 모음입니다. 30 가지 시나리오를 카테고리별로 정리했어요. 본인 상황에 맞는 것만 골라 보면 됩니다.

Chapter 10.1

코드 이해 / 탐색

처음 보는 코드 베이스 30분 만에 적응

> 이 리포의 README · package.json · 핵심 진입점을 읽고, 주니어가 30분 안에 적응할 수 있는 안내문을 작성해 주세요. 한국어, 1.5쪽 분량.

한 함수의 호출 흐름 추적

> apps/api/src/services/user-service.ts 의 createUser 함수가 어디서 호출되는지, 그 위로 4 단계까지 호출 트리를 그려 주세요.

"왜 이렇게 째지?" 추적

> apps/web/src/auth/SessionProvider.tsx 에서 sessionStorage 를 안 쓰고 cookies 만 쓰는 이유를 git log + 관련 PR 본문에서 찾아 주세요.

안 쓰는 코드 찾기

> packages/utils 안의 export 중 다른 곳에서 한 번도 import 되지 않는 함수 목록.

의존성 그래프

> apps/api 와 apps/web 의 packages/* 의존 그래프를 mermaid 로.

Chapter 10.2

리팩터링

한 패턴을 일관되게 적용

> apps/api/src/handlers 에 있는 12개 핸들러 모두를 try/catch 대신 Result 타입 패턴으로 변환. 첫 변환은 보고 동의 받고 진행.

거대한 파일 쪼개기

> apps/api/src/router.ts (1500 줄) 를 도메인별로 4~5 개 파일로 분할. 의미 단위 유지.

타입 보강

> 이 자바스크립트 파일의 핵심 함수 5개에 JSDoc 으로 타입 주석. 같이 import 하는 외부 라이브러리의 공식 타입 정의 참고.

동기 → 비동기

> apps/api/src/db.ts 의 모든 동기 I/O 호출을 async/await 로 변환. 호출 측 함수 시그니처도 함께 변경.

Mocha → Vitest

> 단위 테스트를 Mocha 에서 Vitest 로 마이그레이션. assertion 라이브러리도 chai 에서 vitest 의 expect 로.

Chapter 10.3

디버깅

5xx 의 첫 발생 추적

> Sentry MCP 로 PaymentTimeoutError 의 첫 발생 시점, 그 시점에 머지된 PR, 변경된 파일 목록.

메모리 누수 가설 검증

> apps/api/src/cache 의 setInterval 호출 위치들을 찾아, clearInterval 이 매칭되는지 확인.

느린 쿼리 추적

> Postgres MCP 로 어제 가장 오래 걸린 쿼리 톱 5. 그 중 결제 관련 쿼리의 EXPLAIN 결과.

Heisenbug 디버깅

> "로컬에서는 통과하지만 CI 에서만 실패" 같은 테스트가 있어. tests/integration/payment.test.ts 의 race condition 가능성을 점검.

환경 차이

> 프로덕션과 스테이징의 환경 변수 / Docker 이미지 / Node 버전 차이를 비교 표로.

Chapter 10.4

테스트 작성

함수 하나에 빠른 테스트

> formatCurrency 함수에 대한 단위 테스트 8개. 엡지 케이스 (음수, 0, 1e15) 포함.

통합 테스트 시나리오 도출

> 회원가입 플로우 (이메일 입력 → 인증 → 비밀번호 → 첫 로그인) 의 통합 테스트 5 시나리오. Playwright 로.

Mocking 전략

> 결제 모듈 테스트에서 외부 PG API 를 mocking 하는 전략 3가지 비교 (msw /nock / 실제 sandbox).

Coverage 보고

> 어제 머지된 변경의 커버리지 보고. 미달 영역에 추가할 테스트 5개.

Chapter 10.5

데이터베이스

안전한 마이그레이션

> 기존 users 테이블에 phone 컬럼 추가. 50M 행 기준 다운타임 없이 안전하게:
> 1) NULL 허용으로 추가
> 2) 백필 마이그레이션 (배치)
> 3) NOT NULL 제약 추가
> 각 단계의 SQL + 롤백 SQL 포함.

인덱스 추천

> 어제 가장 느린 쿼리 톱 10 의 EXPLAIN 을 분석해 추천 인덱스 5개.

N+1 쿼리 사냥

> apps/api/src/handlers/order-list.ts 의 코드를 보고 N+1 가능성. 있으면 join 또는 dataloader 로 수정안.

데이터 정합성 체크

> users 와 orders 테이블 간 user_id 외래키 위반 케이스 (orders.user_id 가 존재하지 않는 user 를 가리키는) 을 찾는 SQL.

Chapter 10.6

보안

Secret 스캔

> 이 리포 전체에서 비밀 (DB 비번, API 키, 토큰) 노출 가능성 스캔. .env.example, README, 주석, 테스트 데이터 포함.

권한 우회 가능성

> apps/api 의 라우트 핸들러 중 인증 미들웨어 없이 직접 요청을 받는 것이 있는지 확인.

SQL 인젝션 체크

> 동적 SQL 을 만드는 코드 (string 연결, template literal) 를 모두 찾아 검토.

CORS 설정 검토

> apps/api 의 CORS 설정이 너무 관대한지 확인. allowed origins 가 와일드카드면 위험 표시.

의존성 취약점

> npm audit 결과 중 high/critical 만 정리. 각 항목의 영향과 권장 조치.

Chapter 10.7

성능

번들 크기 분석

> Next.js 의 production 빌드 결과를 보고 가장 큰 청크 5개. 각 청크의 주요 의존성과 줄일 방법 추천.

React 렌더링 최적화

> Profiler 결과를 보고 불필요한 재렌더링이 일어나는 컴포넌트 5개. memo / useMemo / useCallback 적용 추천.

API 응답 캐싱

> apps/api 의 GET 엔드포인트 중 캐싱이 적합한 것 5개. 캐시 키 / TTL / 무효화 전략 제안.

이미지 최적화

> public/ 디렉터리의 이미지 중 webp 로 변환하면 50% 이상 절약될 것 5개. AVIF 도 검토.

Chapter 10.8

문서

README 다시 쓰기

> 현재 README 를 5 분 안에 시작할 수 있도록 다시 정리. 다음 순서: 한 줄 요약 / 빠른 시작 / 디렉터리 / 자주 쓰는 명령 / 기여 가이드.

API 문서 자동 생성

> apps/api 의 라우트들을 OpenAPI 3.1 스펙으로. 응답 예시 포함.

아키텍처 다이어그램

> 우리 시스템의 컴포넌트 + 데이터 흐름 다이어그램을 mermaid 로. 주요 외부 의존성 (DB, Redis, 외부 PG) 포함.

변경 이력

> origin/main 과 origin/release-1.5 사이의 모든 변경을 사용자 관점의 릴리스 노트로. "추가/변경/수정/제거" 4 섹션.

Chapter 10.9

회의 · 소통

미팅 노트 정리

> 첨부한 음성 transcript 를 30 분 회의 결정 + 액션 아이템으로 정리. 각 액션은 담당자 + 마감.

Slack 위클리 공지

> 이번 주 우리 팀이 머지한 PR 들 중 사용자 영향이 있는 것 5개. Slack 마크다운으로 공지 작성.

PR 본문 한국어 한 페이지

> origin/feature/xxx 와 main 의 diff 를 보고, 한국어 한 페이지 PR 본문. 변경/검증/영향 + 리뷰어 호출.

PART 11

자주 묻는 질문

Chapter 11.1

비용

Q. 한 달 사용료는 얼마나 드나요?

가벼운 사이드 프로젝트는 월 0~20 달러, 본격 사용은 월 50~200 달러, 회사 단위는 좌석당 월 30~100 달러 정도입니다. **/cost** 로 매일 모니터링하면서 조절하세요.

Q. 어떤 모델을 써야 하나요?

기본은 Sonnet. 단순 작업 (자동 완성, 짧은 함수) 은 Haiku. 어려운 디자인·디버깅은 Opus. **/model** 로 세션 안에서 즉시 전환됩니다.

Q. 토큰을 가장 빠르게 절약하는 방법?

/compact 자주, MCP 줄이기, 모델 다운그레이드, 한 세션에 한 주제만.

Chapter 11.2

보안

Q. 회사 코드를 외부 LLM 에 보내도 되나요?

회사마다 다릅니다. 정식 도입 전 정보보안팀 / 법무팀에 확인. 많은 회사가 Bedrock / Vertex / Foundry 통합으로 데이터가 자기 클라우드를 떠나지 않도록 설정합니다.

Q. API 키가 유출됐을 때?

즉시 console.anthropic.com 에서 revoke. 새 키 발급. 의심 활동 로그 검토.

Q. Claude 가 실수로 위험한 명령을 실행할 수 있나요?

가능. 그래서 권한 정책을 두는 게 핵심. 처음에는 **/permissions** 의 기본값 (always ask) 으로 시작하고, 본인이 안전하다고 확신하는 명령만 점진적으로 자동 허용으로 추가.

Chapter 11.3

권장 사용

Q. 처음 한 달 동안 무엇부터 해야 하나요?

- 1주차: 설치 + 첫 5분 워크플로 + CLAUDE.md 작성
- 2주차: 메모리·슬래시 명령 활용 + 작은 PR 5개
- 3주차: MCP 1~2개 (GitHub, Slack) 추가
- 4주차: 본인 슬래시 명령 5개 정의 + 권한 정책 다듬기

Q. 책 한 권만 읽으면 끝인가요?

이 가이드북은 입문서입니다. 실전은 본인 프로젝트에서 한 시간씩 만져 봐야 합니다. 이 책의 1/3 만 읽고 50시간 직접 만진 사람이, 책 전체를 읽고 5시간 만진 사람보다 훨씬 잘 합니다.

Q. 다른 도구 (Cursor, Copilot) 와 동시에 쓸 수 있나요?

가능. Claude Code 는 셸 기반이라 IDE 의 Copilot/Cursor 와 충돌하지 않습니다. 셋이 다 있으면 가장 좋은 조합:

- Copilot: 자동 완성
- Cursor: 코드 안에서의 편집 보조
- Claude Code: 큰 작업·여러 파일 변경·셸 작업

Chapter 11.4

학습

Q. 어떤 책 / 강의를 더 봐야 하나요?

공식 문서가 가장 정확. 그 외 추천:

- 이 가이드북의 부록 "학습 리소스" 섹션
- Anthropic 공식 [Cookbook GitHub](#)
- 매일 짧은 한국어 팁: SONSLAB 블로그의 Claude Code 팁 카테고리

Q. 영어 자료 읽기 부담스러워요.

처음에는 부담스럽습니다. Claude 에게 "이 영문 docs 페이지를 한국어로 5분 요약" 부탁하면 한 페이지 → 5 단락 정도로 압축됩니다. 그 요약만 읽고 필요할 때 원문 참조.

Q. 이 책의 다음 갱신은?

매주 일요일 20:00 KST 에 자동 갱신됩니다. SONSLAB 가 운영하며, 새 기능 / 변경 / 사용자 피드백을 매주 반영합니다. 기여하고 싶으면 SONSLAB About 페이지의 이메일로 제안 보내 주세요.

PART 12

프롬프트 템플릿 모음

쓸 수 있는 프롬프트 템플릿 50가지. 본인 워크플로에 자주 등장하는 것을 골라 슬래시 명령으로 만들면 일상이 훨씬 편해집니다.

Chapter 12.1

코드 검토 템플릿

짧은 보안 검토

> 이 PR 의 변경에서 보안 관점 위험 5개. 심각도 (1~5) 와 함께. 한국어 표.

성능 검토

> 이 변경이 운영 환경의 응답 시간 / 메모리 / DB 부하에 미칠 영향. p95, p99 도 추정.

정확성 검토

> 이 PR 의 동작 시나리오 5가지를 손으로 시뮬레이션. 각 시나리오의 예상 결과와 잠재 결함.

컨벤션 검토

> 우리 팀 컨벤션 (CLAUDE.md 참조) 을 위반한 부분 표시. 각 위반에 대한 수정 제안.

Chapter 12.2

작업 분할 템플릿

큰 일감 쪼개기

> 다음 작업을 1시간 단위 6개로 쪼개 주세요. 각 단위는 독립적으로 PR 가능해야 함.

> 작업: <작업 설명>

일정 추정

> 이 영향 분석을 보고 한 명의 시니어가 할 때 / 주니어가 할 때 각각 며칠 걸릴지 추정. 위험 요소 명시.

우선순위

> 다음 5 일감을 (A) 사용자 영향 (B) 구현 난이도 (C) 의존성 기준으로 우선순위 매겨 주세요.

Chapter 12.3

학습 템플릿

새 기술 5분 요약

> <새 기술>의 핵심 개념 5개를 5분 안에 이해할 수 있게 정리. 각 개념마다 한 줄 비유.

코드 베이스 학습 경로

> 이 코드 베이스에 새로 합류한 주니어가 첫 한 달 동안 따라야 할 학습 경로. 매주 학습 목표 + 검증 방법.

인터뷰 준비

> 이 회사 (이 코드 베이스) 의 백엔드 시니어 인터뷰에서 받을 만한 시스템 디자인 질문 5개. 각 질문의 평가 포인트.

Chapter 12.4

운영 템플릿

인시던트 1차 조사

- > 알림: <알림 내용>
- > 다음 절차로 1차 조사:
- > 1) 영향 범위 (어떤 사용자 / 얼마나)
- > 2) 가설 3개 + 각 검증 방법
- > 3) 일시 조치 옵션
- > 4) 에스컬레이션 시점

포스트모템 초안

> 첨부한 인시던트 타임라인을 5whys + blameless 톤의 포스트모템으로. 액션 아이템은 SMART 형식.

주간 보고

> 지난 주 우리 팀이 머지한 PR + 개발 환경 변경 + 운영 인시던트를 한국어 한 페이지로. CTO 가 5분 안에 읽을 수 있도록.

Chapter 12.5

디자인 템플릿

ADR 작성

- > 다음 결정을 ADR 형식으로:
- > 결정: <결정 내용>
- > 옵션 3개 비교 (장단점 / 비용 / 위험)

> 채택 이유 / 반대 의견 / 영향 받는 사람들

시스템 다이어그램

> 우리 시스템의 컴포넌트 + 데이터 흐름 + 외부 의존성을 mermaid 로. 가능하면 시퀀스 + 컴포넌트 두 다이어그램.

API 설계

> <기능> 을 위한 REST API 설계. 다음 항목:

- > - 엔드포인트 + 메서드 + 인증
- > - 요청/응답 스키마
- > - 에러 코드 + 메시지
- > - rate limit
- > - 변경 시 클라이언트 영향

Chapter 12.6

커뮤니케이션 템플릿

사용자 안내문

> 이번 변경 (<변경 내용>) 을 사용자 (비-개발자) 에게 알리는 한국어 안내문. 톤은 친근하지만 정확하게. 3 단락 이내.

영문 PR 코멘트

> 이 한국어 피드백을 외국 동료에게 전달할 영문 PR 코멘트로. 톤은 정중하지만 분명하게.

사과 메일

> 어제 인시던트로 영향 받은 고객에게 사과 + 보상 안내 메일. 한국어, 5 단락 이내. 너무 자기방어적이지 않게.

Chapter 12.7

코드 작성 템플릿

신규 함수 작성

> 다음 시그니처의 함수 작성:

> <시그니처>

> 요구:

- > - 단위 테스트 5개 (정상 + 엣지)
- > - 입력 검증 (zod 스키마)
- > - 에러 처리 (Result 타입)
- > - JSDoc

기존 코드 + 신규 기능

> <파일 경로> 의 기존 패턴을 따라 <새 기능> 추가. 기존 컨벤션 준수 (들여쓰기 / 이름 / 에러 처리 / 테스트).

마이그레이션 SQL

> 다음 스키마 변경의 마이그레이션 SQL:

> <변경 내용>

> 요구:

> - up / down 모두

> - NOT NULL 추가는 백필 분리

> - 50M 행 기준 안전성 코멘트

정규식 작성 / 검증

> 다음 패턴을 매칭하는 정규식:

> <패턴 설명>

> 요구:

> - 패턴 자체 + 5 개 예시 (매칭) + 5 개 반례 (불매칭)

> - 가독성 우선 (라인이 길어도 됨)

Chapter 12.8

자동화 템플릿

셸 스크립트 작성

> 다음 동작의 bash 스크립트:

> <동작>

> 요구:

> - set -euo pipefail

> - 트랩으로 임시 파일 정리

> - 사용법 출력 함수

> - macOS / Linux 둘 다 동작

GitHub Actions 워크플로

> 다음 시나리오의 GH Actions 워크플로:

> - 트리거: <트리거>

> - 단계: <단계 목록>

> - 실패 시 Slack 알림

> - 캐싱 적극 활용

Dockerfile

> 다음 앱의 멀티-스테이지 Dockerfile:

- > 스택: <스택>
- > 요구:
- > - 가능한 한 작은 최종 이미지
- > - non-root 유저
- > - 보안 (apk update --no-cache)
- > - 헬스체크

Chapter 12.9

데이터 분석 템플릿

로그 패턴 추출

- > 첨부한 로그에서 가장 자주 등장한 에러 패턴 5개. 각 패턴의 예시 + 가능한 원인 + 빈도.

메트릭 해석

- > 첨부한 Datadog 메트릭 (지난 24h) 을 보고 이상 징후. 각 이상 징후의 가능 원인.

A/B 테스트 결과 해석

- > 이 A/B 테스트 결과 (Control 1만, Treatment 1만 명) 를 보고 통계적 유의성 + 실용적 유의성 + 권장 행동.

Chapter 12.10

글쓰기 템플릿

블로그 포스트

- > 다음 주제의 블로그 포스트 초안:
- > 주제: <주제>
- > 톤: SONSLAB 톤 (친근, 명확, 자기 경험 1개 포함)
- > 길이: 1500 자 내외
- > 결말: 독자가 내일 시도해 볼 한 가지 행동

컨퍼런스 발표 개요

- > 다음 주제의 30분 발표 개요:
- > 주제: <주제>
- > 청중: 시니어 백엔드 개발자
- > 다음 5 슬라이드 그룹:
- > - Hook (3분)
- > - 본론 1: <소주제 1> (8분)
- > - 본론 2: <소주제 2> (8분)
- > - 본론 3: <소주제 3> (8분)

> - 정리 + Q&A (3분)

Chapter 12.11

회고 템플릿

KPT 형식

- > 이번 스프린트 회고:
- > Keep: 잘 된 것 3개
- > Problem: 문제였던 것 3개
- > Try: 다음에 시도할 것 3개
- > 각 항목은 구체적인 사실 + 한 줄 의미.

팀 1:1 준비

- > 다음 1:1 의 안건 5개를 다음 우선순위로:
- > 1) 본인이 막혀 있는 것
- > 2) 본인이 잘하고 있는 것
- > 3) 팀 / 회사에 대한 피드백
- > 4) 학습 목표
- > 5) 다음 분기 우선순위

PART 13

마무리 — 한 달 동안 만져 보기 가이드

이 책을 끝까지 읽은 본인에게 마지막 선물은 ****계획표****입니다. 한 달 동안 매일 30분만 직접 만지면, 책 한 권을 통째로 머리에 이식한 효과를 손에 가진 효과로 바꿀 수 있어요.

Chapter 13.1

주 학습 계획

1주차 — 손에 익히기 (기본)

월: 설치 + 첫 5분 워크플로 (Part 1.4) 화: 본인 사이드 프로젝트에 **/init** 으로 CLAUDE.md 생성, 다듬기 (Part 1.6, 2.1) 수: 메모리·슬래시 명령 5개 만들기 (Part 2.2~2.3) 목: 본인 코드에서 작은 변경 5개 시키기. 결과 검토 후 commit 금: 일주일 사용 기록을 회고. **/cost** 확인. 본인 작업 패턴 파악 토·일: 휴식 또는 본인 사이드 프로젝트에 자유롭게

2주차 — 도구 늘리기 (중급 입문)

월: GitHub MCP 추가 + 이슈/PR 작업 화: Slack MCP 추가 + 메시지 검색·전송 수: 본인 자주 쓰는 작업 1개를 슬래시 명령으로 만들기 목: Auto mode 로 전환. 한 주간 동작 모니터링 금: **/permissions** 정밀하게 손보기 (allow / deny 각 5개) 토·일: 본인 회사 코드에서 한 작업을 처음부터 끝까지 Claude 와

3주차 — 본격 도입 (중급)

월: Plan 모드로 큰 리팩터링 1개 시작 화: Skills 1개 만들기 (본인 팀 PR 작성 절차) 수: Hook 1개 — 위험 명령 차단 목: 서브에이전트 1개 — 본인 영역 전문가 (DB / API / 프론트) 금: **/team-onboarding** 으로 본인 설정 패키징 토·일: 사이드 프로젝트로 Agent SDK 첫 호출 체험

4주차 — 고급 + 운영 (고급 입문)

월: Agent SDK 로 작은 자동화 (예: 위클리 PR 보고) 화: CI 에 Claude 통합 (PR 자동 리뷰) 수: 본인 모니터링·알림에 Channels 결합 시도 목: 비용 모니터링 + 사용 패턴 정리 금: 본인 한 달 학습 노트 작성. 다음 달 목표 5개 토·일: 휴식

Chapter 13.2

한 달 후의 본인

이 4 주를 따라가면 다음 능력이 손에 납니다.

- 매일 30분 코드 작업의 70% 이상을 Claude Code 안에서 해결
- 큰 리팩터링·마이그레이션을 사고 없이 진행하는 절차감
- 본인만의 슬래시 명령·스킬 묶음 — 일종의 작업 디지털 트윈
- 보안·비용·성능을 항상 의식하면서 도구를 다루는 균형 감각

- 회사 코드 베이스 어디든 30분이면 적응할 수 있는 자신감

Chapter 13.3

추천 다음 단계

이 가이드북을 마쳤다면 다음을 권합니다.

- 매일 SONSLAB 의 [Claude Code 팁 카테고리](#) 한 글 읽기 (오후 11시 발행)
- 매주 일요일 업데이트되는 가이드북 변경 이력 확인 (이 책이 매주 갱신됩니다)
- 한 달에 한 번 본인 슬래시 명령·스킬·MCP 모음 정리·공유
- 분기마다 본인 회사 정보보안팀과 사용 패턴 점검

Chapter 13.4

이 책에 대한 한 마디

이 책은 SONSLAB 가 매주 큐레이션·갱신하는 비공식 한글 가이드북입니다. 공식 자료가 아니라, 매일 직접 써 보면서 가장 유용했던 것들을 한 권으로 묶은 것이에요. 본인에게 도움이 됐다면, 좋은 패턴을 SONSLAB About 페이지의 이메일로 제보해 주세요. 다음 주 갱신에 반영합니다.

읽어 주셔서 감사합니다. 본인의 다음 한 달이 즐거운 만남이 되길 바라며.

— SONSLAB · 손수봉 · shonsubong.github.io/sonslab-blog —

— 본 PDF 는 2026-04-26 (KST) 기준으로 자동 생성되었습니다. SONSLAB —